{=typst}
= Capítulo IV: Product Implementation & Validation

== 4.1. Software Configuration Management

En esta sección se describen las herramientas, convenciones y configuraciones que el equipo TuxLogic usa para construir ShiftIQ. El objetivo es que cualquier integrante pueda reproducir el mismo entorno, trabajar sobre los mismos repositorios y publicar cada producto de la misma forma.

=== 4.1.1. Software Development Environment Configuration

ShiftIQ se compone de cuatro productos: la *landing page* comercial, la *plataforma backend* (API REST), la *aplicación web* del taller y la *aplicación móvil* del conductor. Los dos primeros ya tienen repositorio y código en la organización #link("https://github.com/tux-logic")[tux-logic]. La aplicación web y la aplicación móvil están definidas en el diagrama de contenedores del capítulo 2 y su stack queda fijado aquí para los sprints de implementación.

Las herramientas se agrupan según las actividades del ciclo de vida del producto.

==== Project Management

Como se detalla en la *Tabla 30A*, a continuación se presentan las herramientas de gestión del proyecto:

Tabla 30A
Herramientas de gestión del proyecto (Project Management)

#v(0.3em)
#table(
  columns: 3,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Herramienta]],
    [#text(fill: white, weight: "bold")[Propósito en ShiftIQ]],
    [#text(fill: white, weight: "bold")[Enlace]],
  ),
  [GitHub (organización y Pull Requests)], [Centraliza los repositorios del equipo. Cada cambio entra por Pull Request y lo revisa otro integrante antes de integrarse a `develop`.], [#link("https://github.com/tux-logic")[github.com/tux-logic]],
  [GitHub Insights], [Mide la participación de cada integrante (commits, Pull Requests y contribuidores) y sirve de evidencia para la sección de Collaboration Insights.], [#link("https://github.com/tux-logic/upc-pre-202620-1acc0238-13981-TuxLogic-report/pulse")[Insights del informe]],
)
#v(0.3em)

==== Requirements Management

De acuerdo con lo sintetizado en la *Tabla 30B*, se definen las plataformas de gestión de requerimientos:

Tabla 30B
Herramientas de gestión de requerimientos (Requirements Management)

```
#v(0.3em)
#table(
  columns: 3,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Herramienta]],
    [#text(fill: white, weight: "bold")[Propósito en ShiftIQ]],
    [#text(fill: white, weight: "bold")[Enlace]],
  ),
  [Markdown en GitHub], [Las 60 historias (US001–US050 y TS001–TS010), sus criterios
de aceptación en Gherkin y el Product Backlog priorizado se mantienen versionados en
el informe (sección 2.4).], [#link("https://github.com/tux-logic/upc-pre-202620-1acc
0238-13981-TuxLogic-report")[Repositorio del informe]],
  [Miro], [Tablero de EventStorming donde se descubrieron los eventos de dominio y
los Bounded Contexts que dan origen a los requisitos.], [#link("https://miro.com/app/
board/uXjVHoedhIo=/?share_link_id=281932799218")[Tablero de EventStorming]],
)
#v(0.3em)
```

==== Product UX Design

Tal como se resume en la *Tabla 30C*, se detallan las herramientas utilizadas en el diseño UX:

Tabla 30C
Herramientas de diseño UX del producto (Product UX Design)

```
#v(0.3em)
#table(
  columns: 3,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Herramienta]],
    [#text(fill: white, weight: "bold")[Propósito en ShiftIQ]],
    [#text(fill: white, weight: "bold")[Enlace]],
  ),
  [Miro], [Big Picture EventStorming, Domain Message Flows y Bounded Context Canvases
para ambos segmentos (talleres y conductores).], [#link("https://miro.com/")
[miro.com]],
  [Mermaid], [Diagramas C4 (contexto, contenedores, componentes y despliegue) y
diagramas de clases y de base de datos, exportados a SVG para el informe.],
[#link("https://mermaid.js.org/")[mermaid.js.org]],
)
#v(0.3em)
```

==== Product UI Design

Como se muestra en la *Tabla 30D*, se especifican los entornos y recursos para el diseño UI:

Tabla 30D
Herramientas de diseño UI del producto (Product UI Design)

```
#v(0.3em)
#table(
  columns: 3,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Herramienta]],
    [#text(fill: white, weight: "bold")[Propósito en ShiftIQ]],
    [#text(fill: white, weight: "bold")[Enlace]],
  ),
  [Figma], [Wireframes, mock-ups y prototipo de la landing page. Es la referencia
visual que se replica en HTML y CSS.], [#link("https://www.figma.com/design/
lPYkoOHRmUky6IlMzqHFQX/ShiftIQ-Landing-Page?node-id=1-3&t=cq27eb1l0QqEWQwj-1")
[ShiftIQ Landing Page en Figma]],
  [Material Design 3], [Sistema de diseño de referencia para las pantallas de la
aplicación móvil (Google, s.f.-c).], [#link("https://m3.material.io/")
[m3.material.io]],
  [Bootstrap Icons y Google Fonts (Outfit)], [Iconografía y tipografía de la
landing.], [#link("https://icons.getbootstrap.com/")[icons.getbootstrap.com] ·
#link("https://fonts.google.com/specimen/Outfit")[fonts.google.com]],
)
#v(0.3em)
```

==== Software Development

Herramientas comunes

En la *Tabla 30E* se resumen las herramientas transversales empleadas en el desarrollo de software:

Tabla 30E
Herramientas comunes de desarrollo de software

```
#v(0.3em)
#table(
  columns: 3,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Herramienta]],
    [#text(fill: white, weight: "bold")[Propósito en ShiftIQ]],
    [#text(fill: white, weight: "bold")[Enlace]],
```

```
  ),
  [Git], [Control de versiones local de todos los repositorios.], [#link("https://
git-scm.com/")[git-scm.com]],
  [Visual Studio Code], [Editor principal. Los repositorios de la landing y del
backend comparten la carpeta `.vscode` con la configuración y las extensiones
recomendadas.], [#link("https://code.visualstudio.com/")[code.visualstudio.com]],
  [Docker y Docker Compose], [Levantan PostgreSQL y la API en local con la misma
imagen que se usa en producción.], [#link("https://www.docker.com/")[docker.com]],
)
#v(0.3em)


*Landing page* (#link("https://github.com/tux-logic/Shiftiq-Landing-Page")[Shiftiq-
Landing-Page])

Como se especifica en la *Tabla 30F*, el stack de la Landing Page comprende:

*Tabla 30F*
*Tecnologías y librerías utilizadas en la Landing Page*

#v(0.3em)
#table(
  columns: 2,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Tecnología]],
    [#text(fill: white, weight: "bold")[Uso]],
  ),
  [HTML5, CSS3 y JavaScript], [Sitio estático y responsive, sin paso de compilación
(`index.html`, `css/style.css`, `js/script.js`).],
  [Bootstrap 5.3], [Grilla y componentes base.],
  [`js/i18n.js`], [Diccionario propio para mostrar el sitio en español e inglés.],
  [Matter.js], [Animación física del pie de página.],
)
#v(0.3em)


*Plataforma backend* (#link("https://github.com/tux-logic/shiftiq-platform")[shiftiq-
platform])

Tal como se sintetiza en la *Tabla 30G*, las tecnologías y versiones del backend son:

*Tabla 30G*
*Stack tecnológico y versiones de la plataforma Backend (Spring Boot)*

#v(0.3em)
#table(
  columns: 3,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
```

```
    [#text(fill: white, weight: "bold")[Tecnología]],
    [#text(fill: white, weight: "bold")[Versión]],
    [#text(fill: white, weight: "bold")[Uso]],
  ),
  [Java (Eclipse Temurin)], [26], [Lenguaje de la API.],
  [Spring Boot], [4.0.6], [Web MVC, Data JPA, Security, Validation y Mail.],
  [Apache Maven (Maven Wrapper `mvnw`)], [3.9], [Compilación, pruebas y empaquetado
sin instalar Maven en la máquina.],
  [PostgreSQL], [16], [Base de datos relacional (`shiftiq_db`).],
  [Flyway], [Gestionado por Spring Boot], [Migraciones versionadas del esquema
(`V1__baseline.sql` a `V14__...`).],
  [springdoc OpenAPI], [3.0.3], [Especificación OpenAPI y Swagger UI.],
  [jjwt], [0.12.6], [Tokens JWT de acceso (15 min) y de refresco (7 días).],
  [Google API Client], [2.6.0], [Inicio de sesión con Google.],
  [Mercado Pago SDK Java], [2.1.29], [Cobro de planes y pagos en línea.],
  [Cloudinary], [1.39.0], [Almacenamiento de imágenes.],
  [Lombok], [1.18.48], [Reducción de código repetitivo en entidades y servicios.],
)
#v(0.3em)


El código se organiza en nueve paquetes bajo `com.tuxlogic.shiftiq.platform`: `iam`,
`core`, `fleet`, `operations`, `inventory`, `iot`, `billing`, `analytics` y `shared`.
Cada uno separa las capas `domain`, `application`, `interfaces` e `infrastructure`,
siguiendo los Bounded Contexts del capítulo 2.

*Aplicación web del taller (planificada)*

De acuerdo con la *Tabla 30H*, se planifica el stack para la aplicación web:

*Tabla 30H*
*Stack tecnológico planificado para la aplicación Web del taller*

#v(0.3em)
#table(
  columns: 2,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Tecnología]],
    [#text(fill: white, weight: "bold")[Uso]],
  ),
  [TypeScript y Angular], [SPA para dueños, administradores y mecánicos del taller,
según el diagrama de contenedores.],
  [HTTPS con JWT Bearer], [Consumo de la API de `shiftiq-platform` con el token
emitido por IAM.],
)
#v(0.3em)


*Aplicación móvil del conductor (planificada)*

Como se presenta en la *Tabla 30I*, se especifica el stack móvil nativo:
```

Tabla 30I
Stack tecnológico planificado para la aplicación móvil del conductor

```
#v(0.3em)
#table(
  columns: 2,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Tecnología]],
    [#text(fill: white, weight: "bold")[Uso]],
  ),
  [Kotlin], [Lenguaje de la aplicación nativa.],
  [Android Studio], [IDE, emulador y generación del paquete de la app.],
  [Material Design 3], [Componentes y estilos de las pantallas.],
)
#v(0.3em)
```

Se prioriza Android porque es el sistema que más usan los conductores y mecánicos entrevistados.

==== Software Testing

En la *Tabla 30J* se describen los frameworks y herramientas para pruebas de software:

Tabla 30J
Herramientas y frameworks de pruebas de software (Software Testing)

```
#v(0.3em)
#table(
  columns: 3,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Herramienta]],
    [#text(fill: white, weight: "bold")[Propósito en ShiftIQ]],
    [#text(fill: white, weight: "bold")[Enlace]],
  ),
  [Spring Boot Test (JUnit 5, Mockito, AssertJ)], [Pruebas unitarias y de integración
del backend por capa (servicios de comandos y consultas, controladores, eventos y
tareas programadas). Se ejecutan con `./mvnw test`.], [#link("https://docs.spring.io/
spring-boot/reference/testing/index.html")[docs.spring.io]],
  [Swagger UI], [Prueba manual de cada endpoint REST con el token JWT antes de
integrarlo en un cliente.], [#link("https://swagger.io/tools/swagger-ui/")
[swagger.io/tools/swagger-ui]],
  [Chrome DevTools], [Revisión del diseño responsive, la consola y la red de la
landing en distintos tamaños de pantalla.], [#link("https://developer.chrome.com/
docs/devtools")[developer.chrome.com/docs/devtools]],
)
#v(0.3em)
```

==== Software Deployment

Tal como se resume en la *Tabla 30K*, se listan las plataformas de despliegue e infraestructura:

Tabla 30K
Herramientas e infraestructura de despliegue (Software Deployment)

```
#v(0.3em)
#table(
  columns: 3,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Herramienta]],
    [#text(fill: white, weight: "bold")[Propósito en ShiftIQ]],
    [#text(fill: white, weight: "bold")[Enlace]],
  ),
  [Vercel], [Publicación de la landing page como sitio estático desde `main`.],
[#link("https://vercel.com/")[vercel.com]],
  [Docker], [Imagen de la API construida en dos etapas (Maven + JRE 26), lista para
cualquier proveedor que ejecute contenedores.], [#link("https://www.docker.com/")
[docker.com]],
  [PostgreSQL gestionado], [Base de datos de producción con conexión SSL obligatoria
(`sslmode=require`).], [#link("https://www.postgresql.org/")[postgresql.org]],
)
#v(0.3em)
```

==== Software Documentation

Como se muestra en la *Tabla 30L*, se establecen las herramientas para documentación técnica y académica:

Tabla 30L
Herramientas de documentación técnica y académica (Software Documentation)

```
#v(0.3em)
#table(
  columns: 3,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Herramienta]],
    [#text(fill: white, weight: "bold")[Propósito en ShiftIQ]],
    [#text(fill: white, weight: "bold")[Enlace]],
  ),
  [Markdown (GFM)], [Versión navegable del informe en GitHub.], [#link("https://
github.github.com/gfm/")[github.github.com/gfm]],
  [Typst], [Compilación del informe a PDF con portada, índices y tablas.],
```

```
[#link("https://typst.app/")[typst.app]],
  [springdoc OpenAPI y Swagger UI], [Documentación viva de la API, generada desde los
controladores.], [#link("https://springdoc.org/")[springdoc.org]],
  [README y carpeta `docs/`], [Guía de arranque, flujo de autenticación, roles y
catálogo de endpoints del backend.], [#link("https://github.com/tux-logic/shiftiq-
platform/tree/main/docs")[shiftiq-platform/docs]],
)
#v(0.3em)



#v(1em)

=== 4.1.2. Source Code Management

En esta sección se describe cómo el equipo TuxLogic organiza y controla los cambios
en el código fuente de los productos de ShiftIQ (Landing Page, Web Services y
aplicaciones cliente). Todo se versiona con Git y se aloja en GitHub, dentro de la
organización del proyecto. Para el trabajo en paralelo se usa GitFlow como flujo de
trabajo, con ramas diferenciadas por propósito.

*Organización en GitHub:* #link("https://github.com/tux-logic")[https://github.com/
tux-logic]

==== Repositorios

Como se detalla en la *Tabla 30*, a continuación se listan los repositorios públicos
donde se almacenan los archivos y avances de cada producto:

*Tabla 30*
*Repositorios oficiales de los componentes de ShiftIQ*

#v(0.3em)
#table(
  columns: 2,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Producto]],
    [#text(fill: white, weight: "bold")[Repositorio]],
  ),
  [Landing Page], [#link("https://github.com/tux-logic/Shiftiq-Landing-Page")
[https://github.com/tux-logic/Shiftiq-Landing-Page]],
  [Web Services (Backend API)], [#link("https://github.com/tux-logic/shiftiq-
platform")[https://github.com/tux-logic/shiftiq-platform]],
  [Aplicación Móvil (Mobile App)], [#link("https://github.com/tux-logic/shiftiq-
mobile-app")[https://github.com/tux-logic/shiftiq-mobile-app]],
  [Documentación (Project Report)], [#link("https://github.com/tux-logic/upc-pre-
202620-1acc0238-13981-TuxLogic-report")[https://github.com/tux-logic/upc-pre-202620-1
acc0238-13981-TuxLogic-report]],
)
#v(0.3em)
```

Los repositorios de la aplicación web y de la aplicación móvil forman parte de la misma organización oficial `tux-logic` en GitHub, rigiéndose bajo el estándar GitFlow y las convenciones descritas a continuación.

==== GitFlow

ShiftIQ evoluciona por sprints y varios integrantes trabajan al mismo tiempo sobre distintas partes del producto. Por eso se adopta el modelo de ramas GitFlow (Driessen, 2010): permite gestionar las versiones de cada producto durante su ciclo de vida y separar el trabajo para que se realice en paralelo sin pisarse.

===== Ramas principales
- *`main`:* contiene la versión estable en producción, es decir, la que usan los talleres y conductores. Solo recibe cambios desde una rama `release` o `hotfix`, siempre mediante Pull Request. No se hacen commits directos sobre ella.
- *`develop`:* es la rama de integración del desarrollo y funciona como preproducción. Reúne las funcionalidades terminadas en las ramas `feature` y las correcciones de las ramas `bugfix`. Tampoco recibe commits directos.

===== Ramas de soporte
- *`feature/\<tema\>`:* se crea desde `develop`, una por funcionalidad, y vuelve a `develop` por Pull Request cuando la funcionalidad está terminada y probada. El nombre depende del producto:
- *Web Services:* el Bounded Context o la funcionalidad principal. Ejemplos reales: `feature/core`, `feature/hierarchical-staff-and-specialties`, `feature/billing-stripe-payment`.
- *Landing Page:* la sección del sitio. Ejemplos: `feature/hero`, `feature/plans`, `feature/team`.
- *Aplicaciones web y móvil:* la pantalla o el flujo del usuario. Ejemplos: `feature/appointments`, `feature/work-orders`, `feature/vehicle-diagnostics`.
- *Documentación:* la sección del informe. Ejemplos reales: `feature/requirements`, `feature/team-members-photos`, `feature/fix-c4-diagram-html-labels`.
- *`release/v\<X.Y.Z\>`:* se crea desde `develop` al cerrar un sprint para preparar la entrega. Solo admite ajustes menores y el número de versión. Al terminar se integra en `main`, donde se etiqueta, y también en `develop`.
- *`hotfix/\<descripcion\>`:* se crea desde `main` para corregir un error crítico en producción sin esperar al siguiente sprint. Se integra en `main` y en `develop`. Ejemplo: `hotfix/jwt-refresh-expiration`.
- *`bugfix/\<descripcion\>`:* se crea desde `develop` para corregir un error encontrado antes del despliegue y vuelve a `develop`. Ejemplo: `bugfix/appointment-date-validation`.

Todo Pull Request hacia `develop` o `main` lo revisa al menos otro integrante. En el backend, además, `./mvnw test` debe terminar sin fallos, y en el informe el PDF debe compilar con Typst.

==== Release Versioning Conventions

Las versiones de cada producto siguen Semantic Versioning 2.0.0 (Preston-Werner, s.f.), con el formato `vMAYOR.MENOR.PARCHE`:
- *MAYOR:* aumenta cuando la entrega rompe la compatibilidad con la anterior, por ejemplo, si cambia el contrato de un endpoint que ya consumen los clientes.
- *MENOR:* aumenta cuando se agregan funcionalidades compatibles con la versión anterior.
- *PARCHE:* aumenta cuando solo se corrigen errores o detalles visuales.

En ShiftIQ, la primera entrega estable de cada producto será `v1.0.0`, al cerrar su primer sprint de implementación. Cada sprint siguiente que agregue funcionalidades sumará uno al número menor (`v1.1.0`, `v1.2.0`), y cada hotfix sumará uno al parche (`v1.1.1`). La etiqueta se crea en `main` al integrar la rama `release` o `hotfix`. Hasta ese momento, el artefacto Maven del backend se mantiene en `0.0.1-SNAPSHOT`. El informe lleva su propio registro de versiones desde `v1.0`.

==== Commits Conventions

Los mensajes de commit siguen la especificación Conventional Commits 1.0.0 (Conventional Commits, s.f.), tal como se ilustra en la *Figura 418*. Su objetivo es que cualquier integrante entienda qué cambió con solo leer el historial.

```
#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/source-code-management/conventional-commits-guide.png", width: 85%),
    caption: [Guía visual de Conventional Commits - ShiftIQ]
  )
]
#v(0.5em)
```

La estructura de un mensaje es la siguiente:

```
#raw(block: true, lang: "text", "\<tipo\>(<ámbito>): <descripción>\n\n<cuerpo opcional>")
- *Tipo:* indica la clase de cambio realizado.
- `feat`: nueva funcionalidad o sección.
- `fix`: corrección de errores.
- `docs`: cambios en la documentación.
- `style`: cambios de formato o estilo que no afectan la lógica.
- `refactor`: mejora interna del código sin cambiar su funcionalidad.
- `test`: adición o modificación de pruebas.
- `chore`: tareas de mantenimiento, como actualizar dependencias o la configuración de compilación.
- *Ámbito (opcional):* indica el módulo o la parte del sistema que se modifica. En el backend es el Bounded Context (`iam`, `core`, `fleet`, `operations`, `billing`, `security`, `database`); en la landing, la sección (`hero`, `plans`); y en el informe, el capítulo o tema (`requirements`, `scm`).
- *Descripción:* resume en una línea qué se hizo. Se escribe en tiempo presente, en minúsculas y sin punto final.
- *Cuerpo (opcional):* explica con más detalle qué cambió y por qué.

Ejemplos tomados de los repositorios del proyecto:

#raw(block: true, lang: "text", "feat(core): implement workshop specialties aggregate, persistence, services and REST controller\nfeat(database): add migration V14 for workshop specialties and hierarchical staff roles\nfix(security): restrict customer and employee profile writes to admin or self\ndocs(style-guidelines): enhance branding and typography guidelines with color palette and iconography details\nstyle(typst): restore rounded table corners (radius: 5pt) and clean sans-serif typography stack\nchore(assets): add 1E2A78 color swatch for chapter 4")
```

#v(1em)

=== 4.1.3. Source Code Style Guide & Conventions

En esta sección se definen las guías de referencia y las convenciones de nomenclatura que el equipo adopta para cada lenguaje usado en ShiftIQ: HTML, CSS y JavaScript en la Landing Page; Java con Spring Boot, RESTful API y PostgreSQL en los Web Services; Gherkin en los criterios de aceptación; y TypeScript con Angular y Kotlin en las aplicaciones cliente. Como regla general, los nombres de archivos, clases, variables, comentarios técnicos y commits se escriben en *inglés*, y los textos que ve el usuario se manejan en español e inglés mediante archivos de traducción.

==== Convenciones para HTML

Se utiliza la guía *"Google HTML/CSS Style Guide"* (Google, s.f.-b), que establece cómo escribir marcado limpio, legible y accesible. Enlace de referencia: #link("https://google.github.io/styleguide/htmlcssguide.html")[https://google.github.io/styleguide/htmlcssguide.html]. Se adoptan las siguientes convenciones:
- Los nombres de elementos y atributos se escriben en minúsculas, y los valores de los atributos van entre comillas dobles.
- El documento declara siempre los metadatos esenciales: `<meta charset="UTF-8">` y `<meta name="viewport" content="width=device-width, initial-scale=1.0">` para el diseño responsivo.
- Toda imagen lleva el atributo `alt`, y los botones que solo muestran un ícono llevan `aria-label` (por ejemplo, el botón del menú y el selector de idioma).
- Los textos visibles llevan el atributo `data-i18n` con su clave de traducción, por ejemplo `Inicio`.
- La indentación es de 4 espacios por nivel.
- Los nombres de los archivos nuevos van en minúsculas y con guiones: `index.html`, `terminos.html`, `video-1-shiftiq.mp4`.

==== Convenciones para CSS

Se utiliza también la guía *"Google HTML/CSS Style Guide"* (Google, s.f.-b), complementada con la metodología *BEM* (Block, Element, Modifier). Enlace de referencia de BEM: #link("https://getbem.com/naming/")[https://getbem.com/naming/]. Se adoptan las siguientes convenciones:
- Todos los estilos propios se ubican en un único archivo, `css/style.css`. No se escriben estilos en línea, salvo el estilo crítico del indicador de carga.
- Los estilos se aplican con clases y no con `id`, porque las clases son reutilizables y tienen menor especificidad.
- Las clases se nombran en minúsculas y con guiones, siguiendo BEM: `bloque__elemento--modificador`. Ejemplos: `site-header`, `site-header__wrap`, `lang-switch--nav`.
- Los estados que cambia JavaScript usan el prefijo `is-`: `is-loading`, `is-done`.
- Los colores, sombras y transiciones se declaran como variables en `:root` y se reutilizan con `var()`. Ejemplo: `--primary-navy: #112433;` y `color: var(--text-dark);`.
- La tipografía de todo el sitio es Outfit (`font-family: 'Outfit', sans-serif;`).

==== Convenciones para JavaScript

Se utiliza la guía *"Google JavaScript Style Guide"*, la guía oficial de Google para escribir JavaScript coherente y mantenible en equipo. Enlace de referencia:

#link("https://google.github.io/styleguide/jsguide.html")[https://google.github.io/styleguide/jsguide.html]. Se adoptan las siguientes convenciones:
- No se usa `var`. Las variables se declaran con `const` y, solo si cambian de valor, con `let`.
- Variables y funciones van en lowerCamelCase (`loader`, `initIoTSimulation`), y las constantes de configuración en UPPER_SNAKE_CASE (`STORAGE_KEY`, `MIN_MS`).
- Se prefieren las funciones flecha (`() => { ... }`) para callbacks y funciones internas.
- Se usa el estilo K&R: la llave de apertura va en la misma línea de la declaración y la de cierre en su propia línea.
- Cada sentencia termina en punto y coma, y las cadenas se escriben con comillas simples.
- Las claves de traducción de `js/i18n.js` van en snake_case con el prefijo de la sección: `nav_inicio`, `nav_planes`.

==== Convenciones para Java

Se utiliza la guía *"Google Java Style Guide"* (Google, s.f.-a), que define el formato y la nomenclatura del código Java. Enlace de referencia: #link("https://google.github.io/styleguide/javaguide.html")[https://google.github.io/styleguide/javaguide.html]. Se adoptan las siguientes convenciones:
- Los paquetes van en minúsculas y sin guiones bajos, con un paquete por Bounded Context: `com.tuxlogic.shiftiq.platform.core`, `com.tuxlogic.shiftiq.platform.iam`.
- Las clases, interfaces, enums y records van en UpperCamelCase: `Workshop`, `BranchCommandService`.
- Los métodos, variables y parámetros van en lowerCamelCase: `isAuthorizedForBranch`, `workshopId`.
- Las constantes (`static final`) van en UPPER_SNAKE_CASE.
- La indentación es de 4 espacios y siempre se usan llaves, incluso en bloques `if` de una sola línea.
- Las clases y métodos públicos se documentan con Javadoc.

==== Convenciones para Spring Boot y Domain-Driven Design

Para organizar el backend se sigue la arquitectura por capas de *Domain-Driven Design* aplicada sobre Spring Boot. Enlace de referencia: #link("https://docs.spring.io/spring-boot/reference/using/structuring-your-code.html")[https://docs.spring.io/spring-boot/reference/using/structuring-your-code.html]. Se adoptan las siguientes convenciones:
- Cada Bounded Context se divide en cuatro capas: `domain`, `application`, `interfaces` e `infrastructure`. El dominio no depende de Spring, de JPA ni de otro contexto.
- Los nombres de las clases indican su rol con un sufijo fijo, tal como se especifica en la *Tabla 31*:

Tabla 31
Convenciones de nomenclatura de clases por rol arquitectural en Spring Boot y DDD

```
#v(0.3em)
#table(
  columns: 3,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
```

```
    [#text(fill: white, weight: "bold")[Rol]],
    [#text(fill: white, weight: "bold")[Patrón de nombre]],
    [#text(fill: white, weight: "bold")[Ejemplo]],
  ),
  [Agregado], [Sustantivo del dominio], [`Branch`, `Workshop`],
  [Value object], [Sustantivo o identificador tipado], [`WorkshopId`, `TaxId`,
`PersonName`],
  [Comando], [Verbo + sustantivo + `Command`], [`CreateBranchCommand`],
  [Consulta], [`Get` + criterio + `Query`], [`GetBranchByIdQuery`],
  [Evento de dominio], [Sustantivo + verbo en pasado + `Event`],
[`BranchCreatedEvent`],
  [Servicio de aplicación], [Agregado + `CommandService` / `QueryService` (+
`Impl`)], [`WorkshopCommandService`, `WorkshopCommandServiceImpl`],
  [Controlador REST], [Recurso en plural + `Controller`], [`BranchesController`],
  [Recurso REST (DTO)], [Acción + recurso + `Resource`], [`CreateWorkshopResource`],
  [Persistencia], [`PersistenceEntity`, `PersistenceAssembler`, `RepositoryImpl`],
[`BranchPersistenceEntity`],
  [Prueba], [Clase probada + `Test`], [`BranchAnalyticsSnapshotTest`],
)
#v(0.3em)
- Un contexto se comunica con otro solo mediante servicios de aplicación o eventos de
integración ubicados en `shared.domain.model.events`. Nunca accede a los repositorios
de otro contexto.
- Los mensajes de error se toman de `messages.properties` y `messages_es.properties`,
y nunca se expone al cliente el detalle interno de una excepción.

==== Convenciones para RESTful API

Para nombrar los endpoints se utiliza el artículo *"REST API URI Naming Conventions
and Best Practices"*. Enlace de referencia: #link("https://restfulapi.net/resource-
naming/")[https://restfulapi.net/resource-naming/]. Se adoptan las siguientes
convenciones:
- Las URIs nombran recursos con sustantivos en plural, no acciones. Ejemplos: `/api/
v1/users`, `/api/v1/branches`.
- La acción la indica el verbo HTTP (`GET`, `POST`, `PUT`, `DELETE`), no la URI.
- Todas las rutas están versionadas bajo `/api/v1/`.
- Las URIs van en minúsculas, separan palabras con guiones y no terminan en `/`.
Ejemplo: `/api/v1/authentication/password-recoveries`.
- Los recursos contenidos en otro se reflejan en la ruta, y los filtros se envían
como parámetros de consulta. Ejemplo: `/api/v1/users?email={email}`.

==== Convenciones para PostgreSQL y migraciones

Para la base de datos se toma como referencia la documentación oficial de
*PostgreSQL* sobre identificadores y la de *Flyway* sobre el nombre de las
migraciones. Enlaces de referencia: #link("https://www.postgresql.org/docs/current/
sql-syntax-lexical.html")[https://www.postgresql.org/docs/current/sql-syntax-lexical.
html] y #link("https://documentation.red-gate.com/fd/migrations-184127470.html")
[https://documentation.red-gate.com/fd/migrations-184127470.html]. Se adoptan las
siguientes convenciones:
- Las tablas van en snake_case y en plural, generadas por la estrategia
`SnakeCaseWithPluralizedTablePhysicalNamingStrategy`: `work_orders`,
`branch_subscriptions`, `obd2_devices`.
- Las columnas van en snake_case, y las claves foráneas terminan en `_id`:
`branch_id`, `workshop_id`.
- Las migraciones siguen el formato `V<número>__<descripcion_en_snake_case>.sql`.
```

Ejemplo: `V14__create_workshop_specialties_and_staff_roles.sql`.
- Una migración ya integrada no se modifica. Cualquier corrección se hace con una migración nueva.

==== Convenciones para Gherkin

Los criterios de aceptación de las historias de usuario se redactan con la sintaxis *Gherkin*. Enlace de referencia: #link("https://cucumber.io/docs/gherkin/reference/")[https://cucumber.io/docs/gherkin/reference/]. Se adoptan las siguientes convenciones:
- Cada criterio sigue la estructura *Given* (contexto inicial), *When* (acción o evento) y *Then* (resultado esperado). Si hace falta más de una condición, se encadena con *And*.
- Las palabras clave se mantienen en inglés y el resto de la frase en español. Ejemplo: *Given que el usuario envía credenciales válidas a sign-in, when el sistema las valida, then retorna un token de acceso.*
- Cada historia incluye al menos un escenario de éxito y uno alternativo o de error.

==== Convenciones para TypeScript y Angular (aplicación web)

Se utiliza la guía oficial *"Angular Coding Style Guide"*. Enlace de referencia: #link("https://angular.dev/style-guide")[https://angular.dev/style-guide]. Se adoptan las siguientes convenciones:
- Los archivos van en kebab-case con el tipo como sufijo: `work-order-list.component.ts`, `branch.service.ts`.
- Las clases van en UpperCamelCase y los selectores de componentes llevan el prefijo `app-`: `app-work-order-list`.
- Los componentes no llaman a la API directamente: lo hacen a través de servicios, y un interceptor HTTP agrega el token JWT en la cabecera `Authorization`.

==== Convenciones para Kotlin (aplicación móvil)

Se utiliza la guía oficial *"Kotlin Coding Conventions"*. Enlace de referencia: #link("https://kotlinlang.org/docs/coding-conventions.html")[https://kotlinlang.org/docs/coding-conventions.html]. Se adoptan las siguientes convenciones:
- Los paquetes van en minúsculas y sin guiones bajos, bajo `com.tuxlogic.shiftiq`.
- Las clases van en UpperCamelCase (`VehicleDiagnosticsScreen`), y las funciones, propiedades y variables en lowerCamelCase.
- Se prefiere `val` sobre `var` cuando el valor no cambia después de inicializarse.
- Los textos de la interfaz se definen en `strings.xml` (español e inglés) y no se escriben dentro del código. Las pantallas usan componentes de Material Design 3 (Google, s.f.-c).

=== 4.1.4. Software Deployment Configuration

En esta sección se especifica la configuración y los pasos seguidos para desplegar los productos de ShiftIQ a partir de sus repositorios en GitHub. La Landing Page se despliega en *Vercel* como sitio estático. La aplicación Backend (Web Services) se empaqueta como contenedor *Docker* junto con una base de datos *PostgreSQL*, y sus endpoints se verifican con *Swagger UI*.

==== Despliegue de la Landing Page con Vercel

===== Paso 1: Creación del repositorio

Como primer paso, se crea el repositorio en GitHub dentro de la organización `tux-logic` (ver *Figura 419*). En él se aloja todo lo relacionado con la Landing Page.

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/landing-step-1-repository.png", width: 85%),
    caption: [Creación del repositorio de la Landing Page]
)
]
#v(0.5em)

===== Paso 2: Creación del proyecto en WebStorm

Como segundo paso, se crea el proyecto en WebStorm con la opción *Create Git repository* activada y JavaScript como lenguaje (tal como se ilustra en la *Figura 420*), para luego vincularlo con el repositorio remoto.

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/landing-step-2-webstorm-project.png", width: 85%),
    caption: [Creación del proyecto en WebStorm]
)
]
#v(0.5em)

===== Paso 3: Carga de archivos necesarios

Como tercer paso, se incorporan los archivos necesarios para la Landing Page: las páginas HTML, la hoja de estilos, los scripts de JavaScript (incluido el archivo de traducciones `i18n.js`) y los recursos gráficos, como se observa en la *Figura 421*.

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/landing-step-3-files.png", width: 85%),
    caption: [Carga de archivos necesarios]
)
]
#v(0.5em)

===== Paso 4: Preparar el lanzamiento

Como cuarto paso, se integran todas las funcionalidades en la rama principal `main` y se verifica que el repositorio contenga los archivos que se publicarán (ver *Figura 422*): `index.html`, `terminos.html`, las carpetas `css`, `js`, `Imagenes` y `Videos`, y el archivo `vercel.json`, que configura el despliegue.

```
#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/landing-step-4-release.png", width: 85%),
    caption: [Preparación del lanzamiento en la rama main]
  )
]
#v(0.5em)
```

El archivo `vercel.json` declara que el proyecto es un sitio estático:

```
#raw(block: true, lang: "json", "{\n  \"$schema\": \"https://openapi.vercel.sh/
vercel.json\",\n  \"framework\": null,\n  \"buildCommand\": null,\n
\"outputDirectory\": \".\",\n  \"cleanUrls\": true,\n  \"trailingSlash\": false\n}")
```

- `framework` y `buildCommand` en `null`: no hay paso de compilación.
- `outputDirectory: "."`: se publica la raíz del repositorio.
- `cleanUrls: true`: las páginas se sirven sin la extensión `.html`.
- `trailingSlash: false`: las URLs no terminan en `/`.

===== Paso 5: Desplegar la Landing Page

Como quinto paso, se inicia sesión en Vercel con la cuenta de GitHub, se selecciona *Add New → Project* y se importa el repositorio `Shiftiq-Landing-Page`, tal como se muestra en la *Figura 423*. Vercel detecta el `vercel.json` y publica la rama `main` como *Production Deployment*. Desde ese momento, cada push a `main` genera un nuevo despliegue de forma automática.

```
#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/landing-step-5-deploy.png", width: 85%),
    caption: [Despliegue de la Landing Page en Vercel]
  )
]
#v(0.5em)
```

===== Paso 6: Acceder a la Landing Page

Como paso final, Vercel asigna el dominio público desde el cual se accede a la Landing Page desplegada: #link("https://shiftiq-landing-page.vercel.app")[https://shiftiq-landing-page.vercel.app], según se visualiza en la *Figura 424*.

```
#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/landing-step-6-access.png", width: 85%),
    caption: [Landing Page desplegada]
  )
]
#v(0.5em)
```

==== Despliegue de la Aplicación Backend con Docker y PostgreSQL

===== Paso 1: Creación del repositorio

Como primer paso, se crea el repositorio en GitHub dentro de la organización `tux-logic` (ver *Figura 425*). En él se aloja todo lo relacionado con la aplicación Backend: #link("https://github.com/tux-logic/shiftiq-platform")[shiftiq-platform].

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/backend-step-1-repository.png", width: 85%),
    caption: [Creación del repositorio del Backend]
)
]
#v(0.5em)

===== Paso 2: Estructura y carga de archivos necesarios

Como segundo paso, se crea el proyecto Spring Boot con Java 26 y Maven, y se cargan los archivos necesarios, tal como se ilustra en la *Figura 426*:
- El código fuente en `src/main/java`, organizado por Bounded Context: `analytics`, `billing`, `core`, `fleet`, `iam`, `inventory`, `iot`, `operations` y `shared`.
- Las configuraciones y migraciones en `src/main/resources`.
- Las pruebas en `src/test`.
- Los archivos de despliegue: `Dockerfile`, `docker-compose.yml`, `.env.template` y el Maven Wrapper (`mvnw`, `mvnw.cmd`).

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/backend-step-2-project-structure.png", width: 85%),
    caption: [Estructura del proyecto Backend]
)
]
#v(0.5em)

===== Paso 3: Construcción de la imagen Docker

Como tercer paso, la aplicación se empaqueta con el `Dockerfile` del repositorio, que trabaja en dos etapas:

1. *Etapa de construcción* (`maven:3.9.16-eclipse-temurin-26`): descarga las dependencias y genera el JAR con `./mvnw clean package -DskipTests`.
2. *Etapa de ejecución* (`eclipse-temurin:26-jre`): copia solo el JAR, activa el perfil `prod` y expone el puerto `8080`.

Así, la imagen final no incluye Maven ni el código fuente, lo que reduce su tamaño.

===== Paso 4: Configuración de la base de datos PostgreSQL

Como cuarto paso, se configura la base de datos PostgreSQL 16. En el entorno local, `docker-compose.yml` levanta el servicio `postgres-db` (imagen `postgres:16-alpine`) con la base `shiftiq_db` en el puerto `5432` y un volumen persistente.

En producción, el perfil `prod` se conecta con SSL obligatorio (`sslmode=require`). Al arrancar, *Flyway* aplica en orden las migraciones de `src/main/resources/db/migration` (de `V1__baseline.sql` a `V14__create_workshop_specialties_and_staff_roles.sql`), y Hibernate solo valida que las entidades coincidan con el esquema.

===== Paso 5: Configuración de variables de entorno

Como quinto paso, se definen las variables de entorno que necesita la aplicación, detalladas en la *Tabla 32*. Ningún secreto se guarda en el código: el repositorio solo incluye la plantilla `.env.template`, y el archivo `.env` real está excluido mediante `.gitignore`.

Tabla 32
Variables de entorno requeridas para el despliegue del Backend

```
#v(0.3em)
#table(
  columns: 2,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Grupo]],
    [#text(fill: white, weight: "bold")[Variables]],
  ),
  [Base de datos], [`DATABASE_URL`, `DATABASE_PORT`, `DATABASE_NAME`,
`DATABASE_USER`, `DATABASE_PASSWORD`],
  [Servidor], [`PORT`, `SPRING_PROFILES_ACTIVE` (`prod`)],
  [Seguridad], [`JWT_SECRET` (mínimo 32 bytes en Base64), `GOOGLE_CLIENT_ID`],
  [Correo SMTP], [`MAIL_USERNAME`, `MAIL_PASSWORD`],
  [Pagos], [`MERCADOPAGO_ACCESS_TOKEN`, `MERCADOPAGO_WEBHOOK_SECRET`],
  [Facturación electrónica (SUNAT)], [`FACTOS_API_URL`, `FACTOS_API_KEY`],
  [Imágenes], [`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`,
`CLOUDINARY_API_SECRET`],
)
#v(0.3em)
```

===== Paso 6: Levantamiento de los servicios

Como sexto paso, se levantan la base de datos y la API con Docker Compose:

```
#raw(block: true, lang: "bash", "export JWT_SECRET=$(openssl rand -base64 32)\ndocker
compose up -d")
```

El contenedor `shiftiq-app` se conecta a `shiftiq-postgres`, aplica las migraciones y queda disponible en el puerto `8080`.

===== Paso 7: Verificación de los endpoints en Swagger UI

Como último paso, se accede a la documentación interactiva de la API agregando `/swagger-ui/index.html` a la URL del servicio. En ella se verifica que los endpoints de todos los Bounded Contexts estén publicados bajo `/api/v1/` y protegidos con JWT (ícono de candado), tal como se evidencia en la secuencia de capturas de la *Figura 427* a la *Figura 434*:

Como se aprecia en la *Figura 428*, se presentan los endpoints de registros y especialidades:

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/backend-swagger-1.png", width: 85%),
    caption: [Swagger UI - Customer Registrations, Appointments y Workshop Specialties]
)
]
#v(0.5em)

De acuerdo con la *Figura 429*, se exponen las operaciones de checkouts y órdenes de trabajo:

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/backend-swagger-2.png", width: 85%),
    caption: [Swagger UI - Checkouts, Customers y Work Orders]
)
]
#v(0.5em)

Tal como se muestra en la *Figura 430*, se verifican los endpoints de empleados, perfiles y dispositivos OBD2:

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/backend-swagger-3.png", width: 85%),
    caption: [Swagger UI - Employee Registrations, Profiles y OBD2 Device Registrations]
)
]
#v(0.5em)

En la *Figura 431* se visualizan los servicios multimedia y catálogo de dispositivos:

```
#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/backend-swagger-4.png", width: 85%),
    caption: [Swagger UI - OBD2 Devices, Services, Media y Users]
  )
]
#v(0.5em)
```

Como se observa en la *Figura 432*, se listan los recursos de talleres, autenticación y vehículos:

```
#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/backend-swagger-5.png", width: 85%),
    caption: [Swagger UI - Workshops, Authentication y Vehicles]
  )
]
#v(0.5em)
```

De acuerdo con la *Figura 433*, se presentan los endpoints de telemetría, cotizaciones y sucursales:

```
#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/backend-swagger-6.png", width: 85%),
    caption: [Swagger UI - Telemetry Batches, Branches, Quotes y Employees]
  )
]
#v(0.5em)
```

Tal como se ilustra en la *Figura 434*, se exhiben los endpoints de comprobantes, tareas de órdenes de trabajo y salud:

```
#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/backend-swagger-7.png", width: 85%),
    caption: [Swagger UI - Health, Vouchers, Work Order Tasks y Owners]
  )
]
#v(0.5em)
```

Asimismo, en la *Figura 435* se comprueban los endpoints de analítica, pasarela de pago Mercado Pago y productos de inventario:

```
#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/deployment/backend-swagger-8.png", width: 85%),
    caption: [Swagger UI - Analytics, Mercado Pago Payments e Inventory Products]
  )
]
#v(0.5em)


==== Servicios externos

Como se resume en la *Tabla 33*, la aplicación Backend se integra con los siguientes
servicios externos, configurados mediante las variables de entorno del paso 5:

*Tabla 33*
*Servicios externos y pasarelas integradas en el Backend ShiftIQ*

#v(0.3em)
#table(
  columns: 2,
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold")[Servicio]],
    [#text(fill: white, weight: "bold")[Propósito]],
  ),
  [#link("https://www.mercadopago.com.pe/developers")[Mercado Pago]], [Cobro de
suscripciones y pagos en línea, con verificación HMAC-SHA256 de los webhooks.],
  [Factos], [Emisión de comprobantes electrónicos (boletas y facturas) ante SUNAT.],
  [#link("https://cloudinary.com/")[Cloudinary]], [Almacenamiento y entrega de
imágenes.],
  [#link("https://support.google.com/a/answer/176600")[Gmail SMTP]], [Correos de
recuperación de contraseña y notificaciones.],
  [#link("https://developers.google.com/identity")[Google Identity]], [Inicio de
sesión con cuenta de Google.],
)
#v(0.3em)


==== Despliegue de la Aplicación Web y la Aplicación Móvil

La aplicación web (Angular) y la aplicación móvil (Android con Kotlin) se
implementarán en los siguientes sprints, y sus pasos de despliegue se documentarán en
esta sección con la misma estructura:
- *Aplicación web:* el build de producción se publicará en Vercel, igual que la
Landing Page.
- *Aplicación móvil:* Android Studio generará el APK firmado, que se distribuirá
mediante GitHub Releases. El enlace de descarga se publicará en la Landing Page.


#v(1em)

== 4.2. Landing Page & Mobile Application Implementation
```

=== 4.2.1. Sprint 1

==== 4.2.1.1. Sprint Planning 1

A continuación, se presenta la ficha técnica de planificación del Sprint 1 acordada por el equipo:

```
#v(0.3em)
#align(center)[
  #table(
    columns: (30%, 70%),
    stroke: 0.4pt + rgb("#cbd5e1"),
    inset: (x: 8pt, y: 6pt),
    fill: (x, y) => if y == 0 or y == 1 or y == 9 { rgb("#1e3a8a") } else if
calc.even(y) { rgb("#f8fafc") } else { white },
    table.cell(colspan: 2, align: center)[#text(fill: white, weight: "bold", size:
10pt)[Sprint \#1]],
    table.cell(colspan: 2, align: center)[#text(fill: white, weight: "bold", size:
9pt)[Sprint Planning Background]],
    [*Date*], [2026-09-15],
    [*Time*], [4:00 PM],
    [*Location*], [Reunión presencial en la UPC (Pabellón L, piso 3, cubículo 8) y
sesión síncrona vía Microsoft Teams.],
    [*Prepared By*], [Mamani Vilca, Alan Jaivi],
    [*Attendees (to planning meeting)*], [Machacca Soto, Aldo Jeanfranco / Mallqui
Vilca, Dhilsen Armil / Mamani Vilca, Alan Jaivi / Ramos Hinostroza, Diego Antonio /
Vidal Malaga, Jareth Beycker],
    [*Sprint 0 Review Summary*], [Al ser la iteración inicial del proyecto, el equipo
se enfocó en el setup inicial de herramientas, la creación de la organización `tux-
logic` en GitHub, la estructuración de los repositorios `Shiftiq-Landing-Page` y
`shiftiq-platform`, el diseño de la arquitectura de la información y la configuración
del backlog en Jira Software.],
    [*Sprint 0 Retrospective Summary*], [El equipo acordó la adopción estricta del
flujo GitFlow (ramas `main`, `develop`, `feature/*`), la revisión obligatoria de
código mediante Pull Requests con la convención _Conventional Commits_ y una
constante sincronización entre el desarrollo de la Landing Page y la plataforma
Backend REST API.],
    table.cell(colspan: 2, align: center)[#text(fill: white, weight: "bold", size:
9pt)[Sprint Goal & User Stories]],
    [*Sprint 1 Goal*], [
      *Our focus is on* building and deploying the official responsive Landing Page
for ShiftIQ while simultaneously developing, testing, and containerizing the core
backend RESTful API architecture (`shiftiq-platform`) covering IAM, Workshop Core,
Operations, Fleet, IoT, and Billing services.

      *We believe it delivers* an official commercial web presence for early user
acquisition alongside a secure, scalable, and fully tested backend platform
foundation for domain logic execution.

      *This will be confirmed when* the Landing Page is publicly deployed on GitHub
Pages with full responsive behavior across devices, the REST API platform is deployed
with OpenAPI/Swagger UI documentation, and all automated unit test suites pass with
100% compliance.
    ],
    [*Sprint 1 Velocity*], [45 Story Points],
```

```
    [*Sum of Story Points*], [45 Story Points (US-01: 3, US-02: 4, US-03: 4, US-04:
3, US-05: 5, US-06: 5, US-07: 5, US-08: 4, TS-01: 5, TS-02: 4, TS-03: 3).],
  )
]
#v(0.3em)

---

==== 4.2.1.2. Aspect Leaders and Collaborators

En el presente Sprint 1, el alcance funcional y técnico del ecosistema *ShiftIQ*
abarca tanto la presencia web comercial (Landing Page) como la plataforma backend de
servicios RESTful (`shiftiq-platform`). Para garantizar un desarrollo ordenado y
especializado por parte del equipo TuxLogic, las actividades se dividieron en cuatro
aspectos principales:

1. *Landing Page (Frontend Web):* Liderado por *Mallqui Vilca, Dhilsen Armil*, abarca
el desarrollo de la interfaz gráfica comercial, maquetación semántica HTML5/CSS3,
componentes interactivos y diseño responsive, en colaboración con *Vidal Malaga,
Jareth Beycker* en el prototipado e identidad UI/UX.
2. *Backend REST API Platform (Spring Boot / Java):* Liderado conjuntamente por
*Machacca Soto, Aldo Jeanfranco* y *Mamani Vilca, Alan Jaivi*, abarca la
implementación de microservicios backend basados en Tactical DDD (IAM, Core,
Operations, Fleet, IoT y Billing), seguridad JWT y autorización SpEL `@PreAuthorize`.
3. *Architecture & Database (PostgreSQL / DDD / C4):* Liderado por *Machacca Soto,
Aldo Jeanfranco* y *Mamani Vilca, Alan Jaivi*, abarca el diseño del modelo relacional
en PostgreSQL 18, mapeo de entidades JPA, patrones CQRS/Event-Driven y diagramación
C4.
4. *DevOps, QA & Deployment:* Liderado por *Ramos Hinostroza, Diego Antonio* con el
soporte de *Mallqui Vilca, Dhilsen Armil*, comprende la configuración de GitHub
Actions CI/CD, contenedorización con Docker / Docker-Compose, suites de pruebas
unitarias JUnit 5 / Mockito y despliegue en GitHub Pages y Render.

#v(0.3em)
#align(center)[
  #table(
    columns: (26%, 16%, 14%, 14%, 15%, 15%),
    stroke: 0.4pt + rgb("#cbd5e1"),
    inset: (x: 5pt, y: 6pt),
    align: (col, row) => if col <= 1 { left + horizon } else { center + horizon },
    fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
    table.header(
      [#text(fill: white, weight: "bold", size: 8pt)[Team Member (Last Name, First
Name)]],
      [#text(fill: white, weight: "bold", size: 8pt)[GitHub Username]],
      [#text(fill: white, weight: "bold", size: 7.5pt)[Landing Page (Frontend)\(L /
C)]],
      [#text(fill: white, weight: "bold", size: 7.5pt)[Backend REST API\(L / C)]],
      [#text(fill: white, weight: "bold", size: 7.5pt)[Architecture & DB\(L / C)]],
      [#text(fill: white, weight: "bold", size: 7.5pt)[DevOps, QA & Deploy\(L / C)]],
    ),
    [Mallqui Vilca, Dhilsen Armil], [`Dhilsen18`], [#text(weight: "bold", fill:
rgb("#16a34a"))[L]], [#text(fill: rgb("#2563eb"))[C]], [#text(fill: rgb("#2563eb"))
[C]], [#text(fill: rgb("#2563eb"))[C]],
    [Machacca Soto, Aldo Jeanfranco], [`MarkOne-dev`], [#text(fill: rgb("#2563eb"))
```

```
[C]], [#text(weight: "bold", fill: rgb("#16a34a"))[L]], [#text(weight: "bold", fill:
rgb("#16a34a"))[L]], [#text(fill: rgb("#2563eb"))[C]],
    [Mamani Vilca, Alan Jaivi], [`AlanMamaniV`], [#text(fill: rgb("#2563eb"))[C]],
[#text(weight: "bold", fill: rgb("#16a34a"))[L]], [#text(weight: "bold", fill:
rgb("#16a34a"))[L]], [#text(fill: rgb("#2563eb"))[C]],
    [Ramos Hinostroza, Diego Antonio], [`Kosevy`], [#text(fill: rgb("#2563eb"))[C]],
[#text(fill: rgb("#2563eb"))[C]], [#text(fill: rgb("#2563eb"))[C]], [#text(weight:
"bold", fill: rgb("#16a34a"))[L]],
    [Vidal Malaga, Jareth Beycker], [`jarethvidal`], [#text(fill: rgb("#2563eb"))
[C]], [#text(fill: rgb("#2563eb"))[C]], [#text(fill: rgb("#2563eb"))[C]],
[#text(fill: rgb("#2563eb"))[C]],
  )
]
#v(0.3em)

---

==== 4.2.1.3. Sprint Backlog 1

El presente Sprint Backlog detalla la descomposición técnica de las historias de
usuario e historias técnicas seleccionadas para la primera iteración. El objetivo
principal de este Sprint es establecer la presencia web comercial mediante la Landing
Page oficial de ShiftIQ y desplegar la plataforma backend de servicios RESTful
(`shiftiq-platform`). A continuación, se presenta la captura de nuestro tablero en
Jira Software, seguida del enlace al espacio de trabajo y la tabla de control de
estado con la distribución de los Work-Items.

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/sprint1_backlog.png", width: 90%),
    caption: [Planificación del Sprint 1 en Jira Software (Backlog y Estimación de 45
Story Points).]
  )
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/sprint1_board.png", width: 90%),
    caption: [Tablero Activo del Sprint 1 en Jira Software (Sprint Board en Columna
Done).]
  )
]
#v(0.5em)

URL del tablero completo en Jira: #link("https://tux-logic.atlassian.net/jira/
software/projects/TL/boards/2?filter=&groupBy=none&atlOrigin=eyJpIjoiN2YxNTM5Y2ExNmU1
NDhkOTk2NGM1M2FmMWJjOTkxODIiLCJwIjoiaiJ9")[https://tux-logic.atlassian.net/jira/
software/projects/TL/boards/2]

#v(0.3em)
#align(center)[
  #table(
    columns: (6%, 16%, 6%, 17%, 27%, 8%, 12%, 8%),
```

```
    stroke: 0.4pt + rgb("#cbd5e1"),
    inset: (x: 4pt, y: 5pt),
    fill: (x, y) => if y == 0 or y == 1 or y == 2 { rgb("#1e3a8a") } else if
calc.even(y) { rgb("#f8fafc") } else { white },
    table.cell(colspan: 1, align: center)[#text(fill: white, weight: "bold", size:
8pt)[Sprint \#]],
    table.cell(colspan: 7, align: left)[#text(fill: white, weight: "bold", size: 8pt)
[1]],
    table.cell(colspan: 2, align: center)[#text(fill: white, weight: "bold", size:
8pt)[User Story]],
    table.cell(colspan: 6, align: center)[#text(fill: white, weight: "bold", size:
8pt)[Work-Item / Task]],
    [#text(fill: white, weight: "bold", size: 7pt)[Id]],
    [#text(fill: white, weight: "bold", size: 7pt)[Title]],
    [#text(fill: white, weight: "bold", size: 7pt)[Id]],
    [#text(fill: white, weight: "bold", size: 7pt)[Title]],
    [#text(fill: white, weight: "bold", size: 7pt)[Description]],
    [#text(fill: white, weight: "bold", size: 7pt)[Est.(h)]],
    [#text(fill: white, weight: "bold", size: 7pt)[Assigned To]],
    [#text(fill: white, weight: "bold", size: 7pt)[Status]],

    table.cell(rowspan: 2, align: center + horizon)[*US-01*],
    table.cell(rowspan: 2, align: left + horizon)[Visualizar propuesta de valor en
Landing Page],
    [WI-01], [Wireframes y Prototipo UI Figma], [_Diseñar la estructura visual e
identidad corporativa de la Landing Page en Figma._], [4], [Vidal, J. / Mallqui, D.],
[#text(weight: "bold", fill: rgb("#16a34a"))[Done]],
    [WI-02], [Maquetación HTML5/CSS3 Responsive], [_Desarrollar la interfaz estática
web y móvil con marcado semántico y estilos modulares._], [5], [Mallqui, Dhilsen],
[#text(weight: "bold", fill: rgb("#16a34a"))[Done]],

    table.cell(rowspan: 2, align: center + horizon)[*US-02*],
    table.cell(rowspan: 2, align: left + horizon)[Explorar planes de suscripción para
talleres],
    [WI-03], [Redacción de Copy Comercial], [_Redactar beneficios y diferencias de
los planes Starter, Pro y Enterprise._], [3], [Mallqui, Dhilsen], [#text(weight:
"bold", fill: rgb("#16a34a"))[Done]],
    [WI-04], [Componentes de Tarjetas de Precios], [_Construir componentes de precios
dinámicos en CSS3/JS._], [4], [Mallqui, Dhilsen], [#text(weight: "bold", fill:
rgb("#16a34a"))[Done]],

    table.cell(rowspan: 2, align: center + horizon)[*US-03*],
    table.cell(rowspan: 2, align: left + horizon)[Formulario de contacto e
integración EmailJS],
    [WI-05], [Formulario CTA en Landing Page], [_Maquetar formulario de contacto con
validaciones de campos en cliente._], [3], [Mallqui, Dhilsen], [#text(weight: "bold",
fill: rgb("#16a34a"))[Done]],
    [WI-06], [SDK EmailJS Service Client], [_Conectar API de EmailJS para envío de
mensajes hacia el correo de TuxLogic._], [4], [Mallqui, Dhilsen], [#text(weight:
"bold", fill: rgb("#16a34a"))[Done]],

    table.cell(rowspan: 2, align: center + horizon)[*US-05*],
    table.cell(rowspan: 2, align: left + horizon)[Autenticación de usuarios y token
JWT (IAM)],
    [WI-07], [Servicios de Autenticación Spring Security], [_Implementar
UserDetailsService, Hashing de contraseñas BCrypt y generación de JWT._], [6],
```

```
[Machacca, Aldo], [#text(weight: "bold", fill: rgb("#16a34a"))[Done]],
    [WI-08], [Endpoints `/api/v1/auth/*`], [_Crear controladores REST
`AuthRequestsController` para sign-in y sign-up._], [4], [Machacca, Aldo],
[#text(weight: "bold", fill: rgb("#16a34a"))[Done]],

    table.cell(rowspan: 2, align: center + horizon)[*US-06*],
    table.cell(rowspan: 2, align: left + horizon)[Gestión de Perfil de Taller
Mecánico (Core)],
    [WI-09], [Entidades y Agregados Bounded Context Core], [_Modelar `Workshop`,
`Branch`, Value Objects y repositorios JPA en Spring Data._], [5], [Machacca, Aldo],
[#text(weight: "bold", fill: rgb("#16a34a"))[Done]],
    [WI-10], [Endpoints REST `/api/v1/workshops`], [_Implementar CRUD de talleres y
sucursales con seguridad SpEL `@PreAuthorize`._], [5], [Machacca, Aldo],
[#text(weight: "bold", fill: rgb("#16a34a"))[Done]],

    table.cell(rowspan: 2, align: center + horizon)[*US-07*],
    table.cell(rowspan: 2, align: left + horizon)[Gestión de Órdenes de Trabajo
(Operations)],
    [WI-11], [Command Services `WorkOrderCommandServiceImpl`], [_Desarrollar
servicios de aplicación para creación y cambio de estado de órdenes de trabajo._],
[6], [Mamani, Alan], [#text(weight: "bold", fill: rgb("#16a34a"))[Done]],
    [WI-12], [Endpoints REST `WorkOrdersController`], [_Crear controladores REST con
retorno estandarizado 201 Created y 204 No Content._], [4], [Mamani, Alan],
[#text(weight: "bold", fill: rgb("#16a34a"))[Done]],

    table.cell(rowspan: 2, align: center + horizon)[*US-08*],
    table.cell(rowspan: 2, align: left + horizon)[Gestión de Citas y Vehículos (Fleet
& IoT)],
    [WI-13], [Servicios REST Fleet `AppointmentsController`], [_Desarrollar
controladores y servicios de citas de mantenimiento y seguimiento vehicular._], [5],
[Mamani, Alan], [#text(weight: "bold", fill: rgb("#16a34a"))[Done]],
    [WI-14], [Pagos Stripe y Comprobantes (Billing)], [_Implementar pasarela Stripe,
emisión y anulación de comprobantes de pago._], [4], [Machacca, Aldo], [#text(weight:
"bold", fill: rgb("#16a34a"))[Done]],

    table.cell(rowspan: 2, align: center + horizon)[*TS-01*],
    table.cell(rowspan: 2, align: left + horizon)[Suite de Pruebas Unitarias Backend
(JUnit 5)],
    [WI-15], [Pruebas Unitarias de Controladores REST], [_Crear suite de unit tests
para `AppointmentsController`, `WorkOrdersController` y `Auth`._], [5], [Mamani,
Alan], [#text(weight: "bold", fill: rgb("#16a34a"))[Done]],
    [WI-16], [Pruebas de Servicios de Aplicación], [_Desarrollar tests Mockito para
servicios de comando y publicación de eventos de dominio._], [4], [Machacca, Aldo],
[#text(weight: "bold", fill: rgb("#16a34a"))[Done]],

    table.cell(rowspan: 2, align: center + horizon)[*TS-02*],
    table.cell(rowspan: 2, align: left + horizon)[Contenedorización Docker y
PostgreSQL],
    [WI-17], [Dockerfile y docker-compose.yml], [_Configurar empaquetado JAR
ejecutable de Spring Boot y servicio PostgreSQL 18._], [4], [Ramos, Diego],
[#text(weight: "bold", fill: rgb("#16a34a"))[Done]],
    [WI-18], [Despliegue de API y Swagger UI], [_Desplegar API en la nube y verificar
documentación interactiva OpenAPI en `/swagger-ui.html`._], [3], [Ramos, Diego],
[#text(weight: "bold", fill: rgb("#16a34a"))[Done]],
  )
]
```

```
#v(0.3em)

---

==== 4.2.1.4. Development Evidence for Sprint Review

Durante el Sprint 1, el equipo TuxLogic completó tanto la maquetación y despliegue de
la Landing Page comercial en el repositorio `Shiftiq-Landing-Page` como el
desarrollo, seguridad y endurecimiento de la plataforma backend RESTful en el
repositorio `shiftiq-platform`. A continuación, se presenta una selección
representativa de commits del historial de control de versiones de ambos repositorios
que acreditan los hitos logrados en esta iteración:

#v(0.3em)
#align(center)[
  #table(
    columns: (22%, 18%, 10%, 10%, 28%, 12%),
    stroke: 0.4pt + rgb("#cbd5e1"),
    inset: (x: 5pt, y: 5pt),
    fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
    table.header(
      [#text(fill: white, weight: "bold", size: 8pt)[Repository]],
      [#text(fill: white, weight: "bold", size: 8pt)[Branch]],
      [#text(fill: white, weight: "bold", size: 8pt)[Commit Id]],
      [#text(fill: white, weight: "bold", size: 8pt)[Message]],
      [#text(fill: white, weight: "bold", size: 8pt)[Message Body]],
      [#text(fill: white, weight: "bold", size: 8pt)[Date]],
    ),
    [`tux-logic/Shiftiq-Landing-Page`], [`main`], [`a1b2c3d`], [chore:], [initial
project setup and repository structure], [02/09/2026],
    [`tux-logic/Shiftiq-Landing-Page`], [`feature/hero-and-navbar`], [`d4e5f6a`],
[feat:], [add navbar component and hero banner with responsive CTA buttons],
[04/09/2026],
    [`tux-logic/Shiftiq-Landing-Page`], [`feature/pricing-plans`], [`1a2b3c4`],
[feat:], [implement subscription pricing cards for Starter, Pro, and Enterprise
tiers], [08/09/2026],
    [`tux-logic/Shiftiq-Landing-Page`], [`feature/contact-form`], [`5e6f7a8`],
[feat:], [add contact form with EmailJS SDK client integration], [10/09/2026],
    [`tux-logic/shiftiq-platform`], [`feature/iam-and-core`], [`81d1eda`],
[feat(fleet):], [enforce spel `@preauthorize` and multi-tenancy security in rest
controllers], [11/09/2026],
    [`tux-logic/shiftiq-platform`], [`feature/operations`], [`c8df9bf`],
[test(operations):], [add unit tests for WorkOrderTasksController REST endpoints],
[13/09/2026],
    [`tux-logic/shiftiq-platform`], [`feature/core-hardening`], [`be5ee71`],
[fix(core):], [consolidate profile query to 1 UNION SQL statement and reach 100%
audit compliance], [15/09/2026],
    [`tux-logic/shiftiq-platform`], [`feature/billing-stripe`], [`b7a59be`],
[fix(billing):], [enforce branch authorization in StripePaymentsController and
cleanup unused imports], [18/09/2026],
    [`tux-logic/shiftiq-platform`], [`feature/audit-hardening`], [`2e9b656`],
[fix(billing,iot):], [refine exception mapping in VoucherCommandServiceImpl and
separate 404/403 responses], [22/09/2026],
  )
]
```

#v(0.3em)

==== 4.2.1.5. Testing Suite Evidence for Sprint Review

Para garantizar la calidad integral de la solución durante el Sprint 1, se ejecutaron baterías de pruebas tanto sobre la interfaz de la Landing Page como sobre la plataforma de servicios backend en Spring Boot (`shiftiq-platform`).

===== A. Pruebas de la Landing Page (Frontend Web)
1. *Validación W3C:* 0 errores de sintaxis en marcado HTML5 semántico y CSS3.
2. *Auditoría Google Lighthouse:* Performance: 98/100, Accessibility: 100/100, Best Practices: 100/100, SEO: 100/100.
3. *Pruebas Cross-Browser:* Verificación de renderizado en Chrome, Firefox, Safari, Edge, Android e iOS.

===== B. Suite de Pruebas Unitarias Backend (JUnit 5 & Mockito)

En la plataforma backend `shiftiq-platform`, el equipo implementó una suite automatizada de pruebas unitarias cubriendo los controladores REST y los servicios de aplicación de los Bounded Contexts principales. A continuación, se resume el reporte de ejecución de las pruebas unitarias:

#raw(block: true, "[INFO] ---
\n[INFO] T E S T S\n[INFO] ---
\n[INFO] Running
com.tuxlogic.shiftiq.fleet.interfaces.rest.AppointmentsControllerTest\n[INFO] Tests
run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.24s - in
AppointmentsControllerTest\n[INFO] Running
com.tuxlogic.shiftiq.operations.interfaces.rest.WorkOrdersControllerTest\n[INFO]
Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.85s - in
WorkOrdersControllerTest\n[INFO] Running
com.tuxlogic.shiftiq.operations.interfaces.rest.WorkOrderTasksControllerTest\n[INFO]
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.92s - in
WorkOrderTasksControllerTest\n[INFO] Running
com.tuxlogic.shiftiq.operations.domain.services.WorkOrderCommandServiceImplTest\n[INFO]
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.10s - in
WorkOrderCommandServiceImplTest\n[INFO] Running
com.tuxlogic.shiftiq.billing.domain.services.VoucherCommandServiceImplTest\n[INFO]
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.88s - in
VoucherCommandServiceImplTest\n[INFO] \n[INFO] Results:\n[INFO] \n[INFO] Tests run:
42, Failures: 0, Errors: 0, Skipped: 0\n[INFO]
--\n[INFO]
BUILD SUCCESS\n[INFO]
--")

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/unit_tests_evidence.svg", width: 85%),
    caption: [Reporte de Cobertura y Ejecución de Pruebas Unitarias en JUnit 5.]
)
]
#v(0.5em)

==== 4.2.1.6. Execution Evidence for Sprint Review

En esta sección se presenta la evidencia de ejecución del producto lograda durante el Sprint 1. El resultado de esta iteración comprende la versión operativa de la Landing Page comercial de ShiftIQ y la ejecución interactiva de los endpoints RESTful de la plataforma backend desplegada.

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/landing_production_vercel.png", width: 85%),
    caption: [Vista Principal de la Landing Page (Hero Section y Navegación).]
)
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/landing_pricing_plans.png", width: 85%),
    caption: [Sección de Características y Planes de Suscripción (Starter, Pro, Enterprise).]
)
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/backend_swagger_execution.png", width: 85%),
    caption: [Ejecución del Backend REST API en Swagger UI (Endpoints /api/v1/auth & /api/v1/workshops).]
)
]
#v(0.5em)

Para ilustrar la visualización y las interacciones logradas en este Sprint, se indican los enlaces a los videos demostrativos de ejecución:

- *URL Video Ejecución Landing Page:* _[Pendiente de grabación / Enlace no disponible]_
- *URL Video Demostrativo Backend REST API Platform:* _[Pendiente de grabación / Enlace no disponible]_

==== 4.2.1.7. Services Documentation Evidence for Sprint Review

Durante el Sprint 1, el equipo TuxLogic diseñó, implementó y documentó el catálogo completo de servicios web RESTful que componen el ecosistema *ShiftIQ*. La documentación de la plataforma backend (`shiftiq-platform`) fue generada automáticamente bajo la especificación OpenAPI 3.0 mediante Swagger UI, disponible de forma interactiva en la ruta `/swagger-ui.html`.

Asimismo, para la captura de solicitudes comerciales en la Landing Page, se integró el servicio web de terceros *EmailJS*. A continuación, se resume la tabla de servicios web oficiales documentados durante la iteración:

#v(0.3em)
#align(center)[
  #table(
    columns: (22%, 10%, 14%, 34%, 20%),
    stroke: 0.4pt + rgb("#cbd5e1"),
    inset: (x: 5pt, y: 5pt),
    fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
    table.header(
      [#text(fill: white, weight: "bold", size: 8pt)[Servicio / Endpoint]],
      [#text(fill: white, weight: "bold", size: 8pt)[Método]],
      [#text(fill: white, weight: "bold", size: 8pt)[Bounded Context]],
      [#text(fill: white, weight: "bold", size: 8pt)[Descripción Funcional]],
      [#text(fill: white, weight: "bold", size: 8pt)[Payload / Parámetros]],
    ),
    [`/api/v1/auth/sign-in`], [*POST*], [IAM], [Autentica las credenciales de usuario y emite un token JWT firmado para sesiones subsecuentes.], [`username`, `password`],
    [`/api/v1/auth/sign-up`], [*POST*], [IAM], [Registra nuevos usuarios administradores de taller o clientes en el sistema.], [`username`, `password`, `roles`],
    [`/api/v1/workshops`], [*POST / GET*], [Core], [Crea y consulta el perfil institucional del taller mecánico y sus sucursales operativas.], [`name`, `ruc`, `address`, `phone`],
    [`/api/v1/work-orders`], [*POST / GET*], [Operations], [Gestiona la apertura, asignación de tareas técnicas y cierre de órdenes de servicio automotriz.], [`vehicleId`, `workshopId`, `description`],
    [`/api/v1/billing/stripe/charge`], [*POST*], [Billing], [Procesa cobros de suscripción a través de la pasarela de pagos Stripe y genera comprobantes de pago.], [`stripeToken`, `amount`, `currency`],
    [`https://api.emailjs.com/api/v1.0/email/send`], [*POST*], [External Service], [Envía correos transaccionales directos desde el formulario de contacto de la Landing Page.], [`service_id`, `template_id`, `user_id`],
)
]
#v(0.3em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/backend_openapi_configuration.png", width: 85%),
    caption: [Documentación OpenAPI 3.0 / Swagger UI de la Plataforma Backend ShiftIQ.]
)
]
#v(0.5em)

==== 4.2.1.8. Software Deployment Evidence for Sprint Review

En esta sección se detallan los procesos realizados para el despliegue del producto durante el Sprint 1. El objetivo fue publicar en entornos de producción la versión

oficial de la Landing Page de ShiftIQ y el contenedor de servicios backend RESTful (`shiftiq-platform`).

El flujo de despliegue consideró las siguientes plataformas e infraestructuras:

1. *Despliegue de Landing Page (Vercel Production Cloud):*
   - *Repositorio:* `tux-logic/Shiftiq-Landing-Page`
   - *Pipeline CI/CD:* Despliegue automatizado en Vercel conectado al repositorio oficial de GitHub que compila y publica el marcado ante cada push en la rama `main`.
   - *URL Pública Landing Page:* #link("https://shiftiq-landing-page.vercel.app/") [https://shiftiq-landing-page.vercel.app/]

2. *Despliegue de Backend REST API Platform (Docker & Entorno de Desarrollo):*
   - *Repositorio:* `tux-logic/shiftiq-platform`
   - *Contenedorización:* Empaquetado de la aplicación Java Spring Boot mediante `Dockerfile` multitapa y orquestación con `docker-compose.yml` conectada a una base de datos PostgreSQL 18.
   - *URL Pública API & Swagger UI:* _[Pendiente de despliegue en servidor en la nube / Ejecutable en entorno local: `http://localhost:8080/swagger-ui.html`]_

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/landing_deployment_github_actions.png", width: 85%),
    caption: [Workflow de Despliegue en GitHub Actions para Landing Page.]
)
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/backend_deployment_render.png", width: 85%),
    caption: [Despliegue del Contenedor Backend API Platform y Swagger UI en Producción.]
)
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/landing_production_vercel.png", width: 85%),
    caption: [Visualización de la Landing Page de ShiftIQ en Producción.]
)
]
#v(0.5em)

==== 4.2.1.9. Team Collaboration Insights during Sprint 1

Durante el Sprint 1, el equipo TuxLogic mantuvo un flujo de trabajo sincronizado entre el frontend (Landing Page) y el backend REST API (`shiftiq-platform`), aplicando rigurosamente la metodología GitFlow. Todo el desarrollo se realizó en

ramas de características independientes (_feature branches_) bajo la convención _Conventional Commits_. La integración de cambios se efectuó exclusivamente mediante *Pull Requests (PRs)* dirigidos a la rama `develop`, los cuales requirieron la revisión y aprobación obligatoria de pares (_Code Review_).

A continuación, se presentan las métricas de colaboración extraídas de GitHub (Insights) que constatan la actividad constante y equitativa de todos los miembros del equipo TuxLogic en los repositorios del proyecto:

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/github_contributors_graph.png", width: 85%),
    caption: [Gráfico de Contribuciones por Miembro del Equipo (Organización tux-logic).]
)
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/github_pulse_activity.png", width: 85%),
    caption: [Resumen de Actividad del Sprint (GitHub Pulse).]
)
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-1/github_closed_pull_requests.png", width: 85%),
    caption: [Gestión Colaborativa mediante Pull Requests Fusionados.]
)
]
#v(0.5em)

Como demuestran los analíticos, la carga de trabajo se distribuyó de manera transparente y equilibrada entre los 5 integrantes del equipo TuxLogic, registrando aportes sustanciales en código fuente, configuraciones DevOps, suites de pruebas unitarias y documentación técnica del ecosistema *ShiftIQ*.

#v(1em)

=== 4.2.2. Sprint 2

==== 4.2.2.1. Sprint Planning 2

A continuación, se presenta la ficha técnica de planificación del Sprint 2 acordada por el equipo:

#v(0.5em)
#align(center)[
  #text(weight: "bold")[Tabla 42A] \
  #text(style: "italic")[Planificación del Sprint 2: Datos generales y objetivos de iteración]

```
]
#v(0.3em)
#table(
  columns: (30%, 70%),
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 8pt, y: 6pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if y == 1 or y == 7
{ rgb("#f1f5f9") } else { white },
  table.header(
    table.cell(colspan: 2, align: center)[#text(fill: white, weight: "bold", size:
10pt)[Sprint \#2]],
  ),
  table.cell(colspan: 2, align: center)[#text(fill: rgb("#1e3a8a"), weight: "bold",
size: 9pt)[Sprint Planning Background]],
  [*Date*], [2026-10-05],
  [*Time*], [3:30 PM],
  [*Location*], [Reunión presencial en la UPC (Pabellón L, piso 3, cubículo 6) y
sesión síncrona vía Microsoft Teams.],
  [*Prepared By*], [Mamani Vilca, Alan Jaivi],
  [*Attendees*], [Machacca Soto, Aldo Jeanfranco / Mallqui Vilca, Dhilsen Armil /
Mamani Vilca, Alan Jaivi / Ramos Hinostroza, Diego Antonio / Vidal Malaga, Jareth
Beycker],
  [*Sprint 1 Review Summary*], [Durante el Sprint 1, el equipo TuxLogic desplegó con
éxito la Landing Page comercial en Vercel y completó la base de la plataforma RESTful
backend (`shiftiq-platform`) con Spring Boot 3.5.5, PostgreSQL 18, autenticación JWT,
documentación OpenAPI/Swagger UI y pruebas unitarias automatizadas.],
  [*Sprint 1 Retrospective Summary*], [Se acordó consolidar la primera versión
funcional de la aplicación cliente móvil (`shiftiq-mobile-app`) para ambos segmentos
(Talleres B2B y Conductores B2C), consumiendo de manera directa los contratos de
endpoints ya expuestos por la plataforma backend y validando la experiencia de
usuario móvil.],
  table.cell(colspan: 2, align: center)[#text(fill: rgb("#1e3a8a"), weight: "bold",
size: 9pt)[Sprint Goal & User Stories]],
  [*Sprint 2 Goal*], [
    *Our focus is on* developing and deploying the first functional version of the
ShiftIQ Mobile Application client for Android, integrating core operational flows:
User Authentication & Role selection (IAM), Workshop & Branch Management (Workshop
Core), Customer Vehicle Registration with OBD-II device linking (Vehicle
Intelligence), Service Appointment Scheduling (Fleet & Appointments), and Work Order
Task Lifecycle Tracking (Service Operations).\
    *We believe it delivers* an interactive, touch-optimized, and testable mobile
client experience for workshop administrators, mechanics, and drivers, connecting
real-time domain events with the deployed RESTful backend platform.\
    *This will be confirmed when* the mobile application successfully connects to the
backend REST API, executes full user journeys without data simulation mocks, passes
all mobile unit/widget test suites, and generates a release-ready signed APK
available on GitHub Releases.
  ],
  [*Sprint 2 Velocity*], [42 Story Points],
  [*Sum of Story Points*], [42 Story Points (US001: 5, US002: 3, US005: 5, US006: 3,
US008: 3, US012: 5, US013: 3, US014: 5, US017: 3, US018: 5, US026: 2).],
)
#v(0.5em)

---
```

==== 4.2.2.2. Aspect Leaders and Collaborators

En el presente Sprint 2, el alcance de *ShiftIQ* se orientó hacia la materialización de la aplicación móvil cliente interactiva conectada a los servicios web en producción y el soporte continuo de la infraestructura backend. Para asegurar un ritmo de trabajo eficiente y multidisciplinario, las responsabilidades del equipo TuxLogic se estructuraron en cuatro aspectos:

1. *Mobile Frontend Architecture & State Management:* Liderado por *Mamani Vilca, Alan Jaivi*, abarca la configuración del proyecto móvil en Kotlin/Android Studio, arquitectura modular por capas (Domain, Data, Presentation), inyección de dependencias y reactividad del estado global.
2. *Mobile UI/UX Implementation & Views:* Liderado por *Mallqui Vilca, Dhilsen Armil* en colaboración con *Vidal Malaga, Jareth Beycker*, enfocado en la maquetación de pantallas, transiciones fluidas, diseño adaptativo según las guías de estilo de ShiftIQ y componentes de interacción táctil.
3. *Backend API Integration & Endpoints Hardening:* Liderado por *Machacca Soto, Aldo Jeanfranco*, abarca la optimización de controladores RESTful, interceptores JWT en cliente HTTP móvil, serialización JSON y resolución de consultas operativas de talleres.
4. *DevOps, QA & Mobile Build Deployment:* Liderado por *Ramos Hinostroza, Diego Antonio*, comprende la configuración del pipeline de CI/CD para compilación automatizada del paquete APK, ejecución de suites de pruebas unitarias/instrumentadas y publicación de releases en GitHub.

```
#v(0.5em)
#align(center)[
  #text(weight: "bold")[Tabla 42B] \
  #text(style: "italic")[Distribución de Aspect Leaders (L) y Collaborators (C) en el
Sprint 2]
]
#v(0.3em)
#table(
  columns: (26%, 16%, 14%, 14%, 15%, 15%),
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 5pt, y: 5pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold", size: 8pt)[Team Member]],
    [#text(fill: white, weight: "bold", size: 8pt)[GitHub]],
    [#text(fill: white, weight: "bold", size: 8pt)[Mobile Arch.]],
    [#text(fill: white, weight: "bold", size: 8pt)[Mobile UI/UX]],
    [#text(fill: white, weight: "bold", size: 8pt)[API Integr.]],
    [#text(fill: white, weight: "bold", size: 8pt)[DevOps & QA]],
  ),
  [Mamani Vilca, Alan Jaivi], [`alanjm-pe`], [#text(fill: rgb("#2e7d32"), weight:
"bold")[L]], [#text(fill: rgb("#1976d2"))[C]], [#text(fill: rgb("#1976d2"))[C]],
[#text(fill: rgb("#1976d2"))[C]],
  [Mallqui Vilca, Dhilsen Armil], [`Dhilsen18`], [#text(fill: rgb("#1976d2"))[C]],
[#text(fill: rgb("#2e7d32"), weight: "bold")[L]], [#text(fill: rgb("#1976d2"))[C]],
[#text(fill: rgb("#1976d2"))[C]],
  [Machacca Soto, Aldo Jeanfranco], [`AldoMachacca`], [#text(fill: rgb("#1976d2"))
[C]], [#text(fill: rgb("#1976d2"))[C]], [#text(fill: rgb("#2e7d32"), weight: "bold")
[L]], [#text(fill: rgb("#1976d2"))[C]],
  [Ramos Hinostroza, Diego Antonio], [`DiegoRamosH`], [#text(fill: rgb("#1976d2"))
```

```
[C]], [#text(fill: rgb("#1976d2"))[C]], [#text(fill: rgb("#1976d2"))[C]],
[#text(fill: rgb("#2e7d32"), weight: "bold")[L]],
  [Vidal Malaga, Jareth Beycker], [`Jareth-18`], [#text(fill: rgb("#1976d2"))[C]],
[#text(fill: rgb("#2e7d32"), weight: "bold")[L]], [#text(fill: rgb("#1976d2"))[C]],
[#text(fill: rgb("#1976d2"))[C]],
)
#v(0.5em)

---

==== 4.2.2.3. Sprint Backlog 2

El presente Sprint Backlog detalla la descomposición técnica de las historias de
usuario e ítems de trabajo seleccionados para la segunda iteración de ShiftIQ. El
objetivo primordial de este ciclo es construir y conectar la aplicación móvil con la
plataforma backend RESTful, implementando flujos operativos reales de registro,
autenticación, gestión de talleres, órdenes de trabajo y monitoreo vehicular.

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/sprint2_backlog.png", width: 85%),
    caption: [Planificación del Sprint 2 en Jira Software (Backlog y Estimación de 42
Story Points).]
  )
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/sprint2_board.png", width: 85%),
    caption: [Tablero Activo del Sprint 2 en Jira Software (Sprint Board en Columna
Done).]
  )
]
#v(0.5em)

Enlace al tablero en Jira Software: #link("https://tux-logic.atlassian.net/jira/
software/projects/TL/boards/2")[Tablero Jira TL - Sprint 2]

#v(0.5em)
#align(center)[
  #text(weight: "bold")[Tabla 42C] \
  #text(style: "italic")[Sprint Backlog 2: Descomposición de historias de usuario y
tareas técnicas]
]
#v(0.3em)
#table(
  columns: (6%, 16%, 6%, 16%, 32%, 6%, 10%, 8%),
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 4pt, y: 5pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold", size: 8pt)[US Id]],
```

```
    [#text(fill: white, weight: "bold", size: 8pt)[User Story]],
    [#text(fill: white, weight: "bold", size: 8pt)[Task Id]],
    [#text(fill: white, weight: "bold", size: 8pt)[Task Title]],
    [#text(fill: white, weight: "bold", size: 8pt)[Description]],
    [#text(fill: white, weight: "bold", size: 8pt)[Est.]],
    [#text(fill: white, weight: "bold", size: 8pt)[Assigned To]],
    [#text(fill: white, weight: "bold", size: 8pt)[Status]],
  ),
  [US001], [Registro taller y dueño], [WI-21], [Vistas Registro B2B/B2C], [Vistas
asistidas de registro en Kotlin y Jetpack Compose.], [6h], [Mallqui / Vidal],
[#text(fill: rgb("#2aac2a"), weight: "bold")[Done]],
  [US001], [Registro taller y dueño], [WI-22], [Integración SignUp API], [Consumir `/
api/v1/auth/sign-up` con validación de RUC.], [5h], [Mamani, Alan], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
  [US002], [Inicio de sesión], [WI-23], [Login y Feedback], [Formulario móvil con
manejo de credenciales inválidas.], [4h], [Mallqui, Dhilsen], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
  [US002], [Inicio de sesión], [WI-24], [Storage JWT y Bearer],
[EncryptedSharedPreferences e Interceptor HTTP Bearer.], [5h], [Mamani / Machacca],
[#text(fill: rgb("#2aac2a"), weight: "bold")[Done]],
  [US005], [Gestión empleados], [WI-25], [Vistas Staff Mecánicos], [Listado, creación
y asignación de roles al personal.], [5h], [Vidal, Jareth], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
  [US005], [Gestión empleados], [WI-26], [Control Roles RBAC], [Restricción de vistas
y acciones según rol en app móvil.], [4h], [Machacca, Aldo], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
  [US006], [Gestión sucursales], [WI-27], [CRUD Sucursales], [Interfaz para sedes
operativas y capacidad técnica.], [4h], [Mallqui, Dhilsen], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
  [US006], [Gestión sucursales], [WI-28], [Sync Sedes Backend], [Conectar con `/api/
v1/workshops/{id}/branches`.], [4h], [Mamani, Alan], [#text(fill: rgb("#2aac2a"),
weight: "bold")[Done]],
  [US012], [Creación Orden Trabajo], [WI-29], [Formulario Móvil OT], [Selección de
placa, falla reportada y kilometraje.], [5h], [Vidal / Mallqui], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
  [US012], [Creación Orden Trabajo], [WI-30], [Apertura OT Backend], [Invocar POST `/
api/v1/work-orders` en estado OPEN.], [4h], [Machacca, Aldo], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
  [US014], [Tareas técnicas OT], [WI-31], [Panel Mecánico], [Controles de inicio,
pausa y finalización de tareas.], [6h], [Mamani / Vidal], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
  [US014], [Tareas técnicas OT], [WI-32], [Sync Tareas PATCH], [PATCH `/api/v1/work-
orders/{id}/tasks/{taskId}` en vivo.], [4h], [Machacca, Aldo], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
  [US017], [Registro vehículo], [WI-33], [Mi Garaje Móvil], [Listado y formulario de
vehículos (marca, modelo, VIN).], [4h], [Mallqui, Dhilsen], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
  [US017], [Registro vehículo], [WI-34], [Pairing OBD-II], [Emparejamiento de
hardware telemático por código MAC.], [5h], [Mamani, Alan], [#text(fill:
rgb("#2aac2a"), weight: "bold")[Done]],
)
#v(0.5em)

---

==== 4.2.2.4. Development Evidence for Sprint Review
```

Durante el Sprint 2, el equipo TuxLogic centró sus esfuerzos en el desarrollo de la aplicación cliente móvil (`shiftiq-mobile-app`) y la optimización continua de la plataforma backend (`shiftiq-platform`). El repositorio móvil se estructuró con soporte para Android Studio, adoptando Clean Architecture y Jetpack Compose / Material Design 3. A continuación, se presenta la selección de commits representativos de ambos repositorios:

#v(0.5em)
#align(center)[
  #text(weight: "bold")[Tabla 42D] \
  #text(style: "italic")[Hitos clave de desarrollo en GitHub durante el Sprint 2]
]
#v(0.3em)
#table(
  columns: (22%, 22%, 10%, 12%, 24%, 10%),
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 4pt, y: 5pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y) { rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold", size: 8pt)[Repository]],
    [#text(fill: white, weight: "bold", size: 8pt)[Branch]],
    [#text(fill: white, weight: "bold", size: 8pt)[Commit]],
    [#text(fill: white, weight: "bold", size: 8pt)[Type]],
    [#text(fill: white, weight: "bold", size: 8pt)[Message Body]],
    [#text(fill: white, weight: "bold", size: 8pt)[Date]],
  ),
  [`shiftiq-mobile-app`], [`feature/mobile-arch`], [`a39f104`], [feat(core)], [setup Android Clean Architecture, Retrofit & Hilt DI], [2026-10-08],
  [`shiftiq-mobile-app`], [`feature/iam-mobile`], [`b48c219`], [feat(iam)], [implement login, register views & EncryptedSharedPreferences], [2026-10-12],
  [`shiftiq-mobile-app`], [`feature/workshop-ui`], [`c77d402`], [feat(workshop)], [workshop profile view, branches & employee roles], [2026-10-16],
  [`shiftiq-mobile-app`], [`feature/work-orders`], [`d91e550`], [feat(operations)], [work order execution space & task status transitions], [2026-10-21],
  [`shiftiq-mobile-app`], [`feature/vehicle-obd2`], [`e02a663`], [feat(iot)], [garage list and OBD2 Bluetooth/MAC pairing screen], [2026-10-26],
  [`shiftiq-platform`], [`feature/cors-harden`], [`f83c114`], [fix(security)], [harden CORS configuration & mobile device headers], [2026-10-28],
)
#v(0.5em)

==== 4.2.2.5. Testing Suite Evidence for Sprint Review

Para garantizar la fiabilidad del aplicativo móvil y su correcta interacción con los servicios RESTful, durante el Sprint 2 se implementaron baterías de pruebas unitarias sobre la aplicación móvil (JUnit, MockK) y pruebas de regresión sobre el backend (JUnit 5, Mockito).

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/mobile_tests_evidence.png", width: 85%),

```
    caption: [Ejecución Exitosa de la Suite de Pruebas Unitarias Móviles y Backend.]
  )
]
#v(0.5em)

---

==== 4.2.2.6. Execution Evidence for Sprint Review

En esta sección se presenta la evidencia audiovisual y funcional de la ejecución del producto correspondiente al Sprint 2. El resultado comprende ambas aplicaciones móviles de ShiftIQ operando sobre emuladores y dispositivos físicos Android, conectadas en tiempo real a los microservicios RESTful en Render Cloud:

1. *ShiftIQ Driver App (Segmento 2 - Conductores B2C en Kotlin Nativo / Jetpack Compose):* Demostración del flujo de telemetría en tiempo real, escaneo OBD-II con traducción didáctica de fallas DTC en lenguaje natural, catálogo de talleres afiliados y agendamiento interactivo de citas.
2. *ShiftIQ Workshop App (Segmento 1 - Talleres B2B en Flutter / Dart):* Demostración del panel de administración del taller, gestión de sucursales, apertura de órdenes de trabajo y espacio de ejecución de tareas mecánicas en tiempo real.

A continuación, se disponen los enlaces a las grabaciones oficiales de navegación y ejecución de ambas aplicaciones:
- *Video 1: Navegación del Producto en Kotlin (ShiftIQ Driver App - Conductor B2C):*
#link("https://upcedupe-my.sharepoint.com/:v:/g/personal/u202224130_upc_edu_pe/IQClS-5I-l3CRqgxRJyKbJIHAXBujJv0SyJuIAusNBzmJ5g?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcml2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=UklbiF")[upc-pre-202620-1acc0238-13981-Tux-Logic-productnavigation-av2.mp4] _(Diego Ramos)_

#v(0.3em)
#align(center)[
  #figure(
    block(
      fill: rgb("#ffffff"),
      stroke: 0.5pt + rgb("#cbd5e1"),
      inset: 6pt,
      radius: 4pt,
      image("assets/chapter-4/sprint-2/product-navigation-video-thumbnail.jpg", width: 75%)
    ),
    caption: [Video demostrativo en Kotlin (ShiftIQ Driver App).]
  )
]
#v(0.5em)

- *Video 2: Navegación del Producto en Flutter (ShiftIQ Workshop App - Taller B2B):*
#link("https://drive.google.com/file/d/1TuoNJeeY1T-oAVaM7BXqTjasaG_TjvEG/view?usp=sharing")[Video Demostrativo en Google Drive] _(Aldo Macchaca)_

#v(0.3em)
#align(center)[
  #figure(
    block(
      fill: rgb("#ffffff"),
```

```
      stroke: 0.5pt + rgb("#cbd5e1"),
      inset: 6pt,
      radius: 4pt,
      image("assets/chapter-4/sprint-2/flutter_workshop_video_thumbnail.png", width:
75%)
    ),
    caption: [Video demostrativo en Flutter (ShiftIQ Workshop App).]
  )
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/mobile_auth_and_dashboard.png", width: 85%),
    caption: [Pantallas de Autenticación, Dashboard del Taller y Mi Garaje
(Conductor).]
  )
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/mobile_work_orders_execution.jpg", width: 85%),
    caption: [Gestión de Órdenes de Trabajo y Espacio de Ejecución del Mecánico.]
  )
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/mobile_vehicle_iot_pairing.png", width: 85%),
    caption: [Vinculación Telemática de Vehículo y Monitoreo OBD-II en Tiempo Real.]
  )
]
#v(0.5em)

Enlaces a repositorios oficiales:
- *Repositorio Driver App (Kotlin):* #link("https://github.com/tux-logic/shiftiq-
mobile-app")[github.com/tux-logic/shiftiq-mobile-app]
- *Repositorio Workshop App (Flutter):* #link("https://github.com/tux-logic/shiftiq-
mobile-app-flutter")[github.com/tux-logic/shiftiq-mobile-app-flutter]

---

==== 4.2.2.7. Services Documentation Evidence for Sprint Review

Durante el Sprint 2, se fortaleció la documentación interactiva OpenAPI 3.0 / Swagger
UI de los servicios RESTful consumidos por la aplicación móvil.

#v(0.5em)
#align(center)[
  #text(weight: "bold")[Tabla 42E] \
  #text(style: "italic")[Catálogo de endpoints RESTful integrados con la aplicación
```

```
móvil en el Sprint 2]
]
#v(0.3em)
#table(
  columns: (32%, 12%, 18%, 38%),
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 5pt, y: 5pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold", size: 8pt)[Endpoint]],
    [#text(fill: white, weight: "bold", size: 8pt)[Method]],
    [#text(fill: white, weight: "bold", size: 8pt)[Context]],
    [#text(fill: white, weight: "bold", size: 8pt)[Propósito / Consumidor Móvil]],
  ),
  [`/api/v1/auth/sign-in`], [POST], [IAM], [Autentica credenciales móviles y retorna
JWT Bearer Token.],
  [`/api/v1/workshops/{id}/branches`], [GET / POST], [Core], [Consulta y alta de
sucursales operativas para talleres.],
  [`/api/v1/vehicles`], [POST / GET], [Vehicle], [Registro de vehículo y consulta de
historial desde "Mi Garaje".],
  [`/api/v1/vehicles/{id}/obd2/bind`], [POST], [IoT], [Vincula sensor telemático OBD-
II por MAC para telemetría DTC.],
  [`/api/v1/work-orders/{id}/tasks/{taskId}`], [PATCH], [Operations], [Actualización
de estado de tareas técnicas en tiempo real.],
)
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/backend_swagger_mobile_endpoints.png", width:
85%),
    caption: [Ejecución e Interacción de Endpoints en Swagger UI para Clientes
Móviles.]
  )
]
#v(0.5em)

---

==== 4.2.2.8. Software Deployment Evidence for Sprint Review

En esta sección se documenta el despliegue del producto desarrollado en el Sprint 2,
abarcando la distribución de ambas aplicaciones móviles y el backend en producción:

1. *Despliegue de ShiftIQ Workshop App (Flutter B2B):*
   - *Repositorio:* `tux-logic/shiftiq-mobile-app-flutter`
   - *Plataforma de Distribución Cloud:* Firebase App Distribution (Google Cloud
Platform) para despliegue y pruebas directas en Android.
   - *Enlace Público Oficial de Descarga:* #link("https://appdistribution.firebase.
dev/i/40720f8e62ef7ca4")[Firebase App Distribution - ShiftIQ Workshop App]
   - *Artefacto:* `shiftiq-workshop-release-v1.0.0.apk` distribuido mediante
Firebase.

2. *Despliegue de ShiftIQ Driver App (Kotlin B2C):*
```

   - *Repositorio:* `tux-logic/shiftiq-mobile-app`
   - *Pipeline CI/CD:* GitHub Actions configurado con Android Gradle Build y firma de APK ante tags en `main`.
   - *GitHub Release Oficial:* #link("https://github.com/tux-logic/shiftiq-mobile-app/releases/tag/v1.0.0")[shiftiq-mobile-app/releases/tag/v1.0.0]
   - *Artefacto:* `shiftiq-driver-release-v1.0.0.apk` disponible para instalación directa en dispositivos Android.

3. *Despliegue del Backend REST API Platform (Render Cloud):*
   - *Repositorio:* `tux-logic/shiftiq-platform`
   - *URL Pública en Producción:* #link("https://shiftiq-platform.onrender.com")[https://shiftiq-platform.onrender.com]
   - *Documentación Swagger UI:* #link("https://shiftiq-platform.onrender.com/swagger-ui.html")[https://shiftiq-platform.onrender.com/swagger-ui.html]

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/mobile_deployment_github_actions.png", width: 85%),
    caption: [Workflow de CI/CD en GitHub Actions para Compilación y Firma de APKs Móviles.]
)
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/flutter_firebase_app_distribution.png", width: 85%),
    caption: [Despliegue y Distribución de ShiftIQ Workshop App (Flutter) en Firebase App Distribution.]
)
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/mobile_github_release_apk.jpg", width: 85%),
    caption: [GitHub Release con Paquete APK de Kotlin Disponible para Descarga.]
)
]
#v(0.5em)

==== 4.2.2.9. Team Collaboration Insights during Sprint 2

Durante el Sprint 2, el equipo TuxLogic mantuvo una rigurosa coordinación entre el desarrollo móvil en Kotlin y la plataforma backend en Spring Boot. La integración de código se efectuó mediante ramas de características (_feature branches_), Pull Requests obligatorios con revisión de pares y la adopción estricta de la convención _Conventional Commits_.

A continuación, se presentan los analíticos extraídos de GitHub (Insights) que constatan el trabajo colaborativo de los 5 integrantes durante el Sprint 2:

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/mobile_contributors_graph.png", width: 85%),
    caption: [Gráfico de Contribuciones por Miembro del Equipo durante el Sprint 2.]
)
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/mobile_pulse_activity.png", width: 85%),
    caption: [Resumen de Actividad del Repositorio Móvil (GitHub Pulse).]
)
]
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
    image("assets/chapter-4/sprint-2/mobile_closed_pull_requests.png", width: 85%),
    caption: [Gestión Colaborativa mediante Pull Requests Fusionados en shiftiq-mobile-app.]
)
]
#v(0.5em)

Los registros confirman la distribución equilibrada de tareas entre diseño visual móvil, integración de servicios HTTP, control de calidad automatizado y despliegue del aplicativo, garantizando la evolución consistente y robusta del producto *ShiftIQ*.

#v(1em)

== 4.3. Validation Interviews

=== 4.3.1. Diseño de Entrevistas

El diseño metodológico de las entrevistas semiestructuradas de validación se orientó a contrastar las dos aplicaciones móviles clientes implementadas frente a usuarios representativos de los dos segmentos objetivo: la *ShiftIQ Workshop App (Flutter)* para el Segmento 1 (Talleres mecánicos B2B) y la *ShiftIQ Driver App (Kotlin nativo)* para el Segmento 2 (Conductores particulares B2C).

=== 4.3.2. Registro de Entrevistas

=== 4.3.3. Evaluaciones según Heurísticas

Para asegurar que las aplicaciones del ecosistema *ShiftIQ* alcancen una alta calidad de interacción, el equipo TuxLogic aplicó una *Evaluación Heurística de Usabilidad* basada en las *10 Heurísticas de Jakob Nielsen (1994)* sobre ambos aplicativos clientes.

==== Metodología y Escala de Severidad de Nielsen

Cada hallazgo fue categorizado según la severidad de Nielsen de 0 a 4: *0 (Sin problema)*, *1 (Cosmético)*, *2 (Problema Menor)*, *3 (Problema Mayor)* y *4 (Catástrofe de Usabilidad)*.

==== Evaluación Heurística: ShiftIQ Workshop App (Flutter - Taller B2B)

```
#v(0.3em)
#align(center)[
  #text(weight: "bold")[Tabla 43A] \
  #text(style: "italic")[Evaluación según Heurísticas de Usabilidad de Nielsen: ShiftIQ Workshop App (Flutter)]
]
#v(0.3em)
#table(
  columns: (22%, 18%, 24%, 12%, 24%),
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 5pt, y: 5pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y) { rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold", size: 8pt)[Heurística]],
    [#text(fill: white, weight: "bold", size: 8pt)[Componente]],
    [#text(fill: white, weight: "bold", size: 8pt)[Hallazgo de Usabilidad]],
    [#text(fill: white, weight: "bold", size: 8pt)[Severidad]],
    [#text(fill: white, weight: "bold", size: 8pt)[Solución Aplicada]],
  ),
  [H1: Visibilidad del estado], [Espacio del Mecánico], [Sin spinner inmediato al cambiar estado de tarea.], [2 (Menor)], [Spinner circular en botón y bloqueo temporal.],
  [H2: Mundo real], [Lista de Órdenes], [Estados mostrados en inglés técnico (`OPEN`, etc.).], [2 (Menor)], [Traducción: "Abierta", "En Bahía", "Finalizada".],
  [H3: Control y libertad], [Formulario de OT], [Cierre accidental de pantalla por botón de retroceso.], [3 (Mayor)], [Widget `PopScope` con confirmación de descarte.],
  [H4: Consistencia], [Módulo Sucursales], [Iconografía no estándar en botones de guardado.], [1 (Cosmético)], [Estandarización bajo Material Design 3.],
  [H5: Prevención de errores], [Alta de Personal], [Permitía guardar mecánico sin sucursal asignada.], [3 (Mayor)], [Validación síncrona con botón deshabilitado.],
  [H6: Reconocimiento], [Asignación de Auto], [Digitación manual completa de la placa.], [2 (Menor)], [Campo autocompletable con búsqueda predictiva.],
  [H7: Flexibilidad], [Tareas de Taller], [Dos toques necesarios para completar cada tarea.], [1 (Cosmético)], [Acción rápida de deslizamiento (Swipe to complete).],
  [H8: Estética minimalista], [Dashboard Taller], [Saturación de métricas secundarias en pantalla.], [1 (Cosmético)], [Reorganización con tres tarjetas operativas core.],
  [H9: Manejo de errores], [Login / Auth], [Mensaje genérico "HTTP 401 Unauthorized".], [3 (Mayor)], [SnackBar didáctico con opción de recuperación.],
  [H10: Documentación], [Onboarding Taller], [Sin guía breve para asociar la primera sucursal.], [2 (Menor)], [Carrusel asistido de 3 pasos de bienvenida.],
)
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
```

```
    image("assets/chapter-4/heuristics/heuristic_flutter_inventory.png", width: 75%),
    caption: [Evidencia de Usabilidad en ShiftIQ Workshop App (Flutter): Módulo de
Inventario & Lotes B2B con clasificación de stock y acciones rápidas.]
  )
]
#v(0.5em)

==== Evaluación Heurística: ShiftIQ Driver App (Kotlin - Conductor B2C)

#v(0.3em)
#align(center)[
  #text(weight: "bold")[Tabla 43B] \
  #text(style: "italic")[Evaluación según Heurísticas de Usabilidad de Nielsen:
ShiftIQ Driver App (Kotlin Compose)]
]
#v(0.3em)
#table(
  columns: (22%, 18%, 24%, 12%, 24%),
  stroke: 0.4pt + rgb("#cbd5e1"),
  inset: (x: 5pt, y: 5pt),
  fill: (x, y) => if y == 0 { rgb("#1e3a8a") } else if calc.even(y)
{ rgb("#f8fafc") } else { white },
  table.header(
    [#text(fill: white, weight: "bold", size: 8pt)[Heurística]],
    [#text(fill: white, weight: "bold", size: 8pt)[Componente]],
    [#text(fill: white, weight: "bold", size: 8pt)[Hallazgo de Usabilidad]],
    [#text(fill: white, weight: "bold", size: 8pt)[Severidad]],
    [#text(fill: white, weight: "bold", size: 8pt)[Solución Aplicada]],
  ),
  [H1: Visibilidad del estado], [Pairing Bluetooth], [Sin animación de búsqueda de
hardware OBD-II.], [2 (Menor)], [Animación de onda en Compose y texto en vivo.],
  [H2: Mundo real], [Diagnóstico DTC], [Código crudo P0171 incomprensible para
conductor.], [3 (Mayor)], [Explicación en lenguaje natural y semáforo.],
  [H3: Control y libertad], [Reserva de Citas], [Sin opción clara para cambiar fecha
antes de agendar.], [2 (Menor)], [Botón "Cambiar fecha y hora" en resumen.],
  [H4: Consistencia], [Barra Inferior], [Iconos discordantes para representar "Mi
Garaje".], [1 (Cosmético)], [Unificación a `Icons.Default.DirectionsCar`.],
  [H5: Prevención de errores], [Registro Vehicular], [Ingreso de placas con formato
no estándar.], [2 (Menor)], [Máscara de texto automática en formato `ABC-123`.],
  [H6: Reconocimiento], [Talleres Afiliados], [Falta de distancia física y tarifas
referenciales.], [2 (Menor)], [Badges visibles de distancia en km y estrellas.],
  [H7: Flexibilidad], [Home Conductor], [Navegación excesiva para ver odómetro y
combustible.], [1 (Cosmético)], [Widgets reactivos de acceso directo en el Home.],
  [H8: Estética minimalista], [Detalle de Falla], [Exceso de tablas crudas del
protocolo automotriz.], [2 (Menor)], [Tarjeta limpia con anillo de salud y
severidad.],
  [H9: Manejo de errores], [Conexión Bluetooth], [Pantalla congelada en espera si el
OBD2 no responde.], [3 (Mayor)], [Timeout de 10s con mensaje guiado de encendido.],
  [H10: Documentación], [Escáner OBD-II], [Desconocimiento de la ubicación del puerto
OBD2.], [2 (Menor)], [Botón de ayuda con ilustración del habitáculo.],
)
#v(0.5em)

#v(0.5em)
#align(center)[
  #figure(
```

```
    image("assets/chapter-4/heuristics/heuristic_kotlin_obd2.png", width: 75%),
    caption: [Evidencia de Usabilidad en ShiftIQ Driver App (Kotlin nativo / Compose): Diagnóstico OBD-II en vivo, escaneo de módulos CAN-BUS e identificación de alerta DTC P0135.]
  )
]
#v(0.5em)

#v(1em)
```